



**COMSATS University Islamabad Abbottabad,
Pakistan**

Turbo Track Rivals (3D Racing Game)

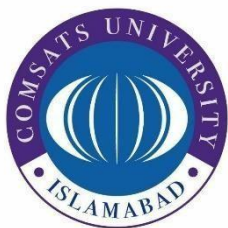
By

Zakariya Anwar Khan	CIIT/FA21-BSE-042/ATD
Muhammad Asim	CIIT/FA21-BSE-025/ATD
Hanif – Ud - Din	CIIT/FA21-BSE-013/ATD

Supervisor

Dr. Muhammad Ibtisam Gul

Bachelor of Science in Computer Science (2021-2025)



COMSATS University, Islamabad Pakistan

Turbo Track Rivals (3D Racing Game)

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment of the requirement for the degree of

Bachelor of Science in Computer Science (2021-2025)

By

Muhammad Asim CIIT/FA21-BSE-025/ATD

Zakariya Anwar Khan CIIT/FA21-BSE-042/ATD

Hanif – Ud - Din CIIT/FA21-BSE-013/ATD

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Zakariya Anwar Khan

Muhammad Asim

Hanif - Ud - Din

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) **Turbo Track Rivals (3D Racing Game)** was develop by **Muhammad Asim (CIIT/FA21-BSE-025/ATD) , Hanif-Ud-Din (CIIT/FA21-BSE-013/ATD) and Zakariya Anwar Khan (CIIT/FA21-BSE-042/ATD)** under the supervision of **Dr. Muhammad Ibtisam Gul** and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Internal Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

Turbo Track Rivals is an exciting and immersive racing game developed in Unity, offering a blend of realistic vehicle mechanics and arcade-style gameplay. The game provides players with the thrill of racing through diverse and challenging tracks, including Stadium, Desert Dunes, and Snow Tracks, while competing against both AI opponents and time. It features two primary modes: Sprint Mode, a time-trial challenge, and Circuit Mode, which puts players against AI controlled cars in competitive races.

The game uses Unity's physics engine to simulate realistic vehicle behavior, including acceleration, braking, and collisions, while integrating dynamic elements like that add layers of strategy to each race. Opponent AI is even tailored to showcase smart and competitive behavior on the race, and it modifies tactics according to the actions of the player and in-game circumstances. The major details of Turbo Track Rivals are two types of games (Sprint Mode and Circuit Mode), several theme tracks (Stadium, Desert Dunes, and Snow Tracks), realistic vehicle physics and Unity physics engine, opponents with different driving styles and difficulties and individualized customization (unlocked vehicles and track options along the way). The project will be designed as a modular design approach with emphasis on reusability and scalability to cater to future expansions. The building is based on an iterative SDLC approach, which makes it possible to provide feedback and refining after testing and user experiences. The games Turbo Track Rivals is targeted to offer an entertaining, thrilling, and enjoyable racing experience to both casual gamers and the serious racing community.

ACKNOWLEDGEMENT

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **Dr. Muhammad Ibtisam Gul**. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Zakariya Anwar Khan

Muhammad Asim

Hanif - Ud - Din

ABBREVIATIONS

SRS	Software Require Specification
SDD	Software Design Document
AI	Artificial intelligence
HUD	Heads Up Display
UI	User Interface
HDRP	High Definition Render Pipeline
UML	Unified Modeling Language
JSON	JavaScript Object Notation
WASD	Keyboard Key: W, A, S, D
FR	Functional Requirements
NFR	Non-Functional Requirements

TABLE OF CONTENT

Table of Contents

1. INTRODUCTION	1
1.1 BRIEF	1
1.2 RELEVANCE TO COURSE MODULES	1
1.3 PROJECT BACKGROUND	2
1.4 LITERATURE REVIEW	2
1.5 ANALYSIS FROM LITERATURE REVIEW	3
1.6 METHODOLOGY AND SOFTWARE LIFECYCLE FOR THIS PROJECT	4
1.6.1 Rationale behind Selected Methodology (OOA)	4
1.6.2 Rationale behind Selected Methodology (Iterative)	4
2. PROBLEM DEFINITION	5
2.1 PROBLEM STATEMENT	5
2.2 DELIVERABLE AND DEVELOPMENT REQUIREMENTS	5
2.3 CURRENT SYSTEM (IF APPLICABLE TO YOUR PROJECT)	7
3. REQUIREMENT ANALYSIS	8
3.1 STORYBOARD	9
3.1.1 Storyboard description	10
3.2 FUNCTIONAL REQUIREMENTS	10
3.3 NON-FUNCTIONAL REQUIREMENTS	12
4. DESIGN AND ARCHITECTURE	13
4.1 SYSTEM ARCHITECTURE	13
4.1.1 Onion Architecture (Turbo Track Rivals)	13
4.1.2 Core Mechanics (Center Layer)	14
4.1.3 Secondary Mechanics (Second Layer)	14
4.1.4 Profile Layer (Third Layer)	15
4.1.5 Narration Layer (Outer Layer)	15
4.2 PROCESS FLOW/REPRESENTATION	15
4.2.1. Description Activity diagram	16
4.3 DESIGN MODELS [ALONG WITH DESCRIPTIONS]	17
4.3.1 Class Diagram	17
4.3.2 Class Diagram Description (Turbo Track Rivals)	19
4.3.3 Key Classes and Their Roles:	19
4.3.4 Relationships and Interactions:	20
4.3.5 Purpose of the Class Diagram:	20
4.3.6 Sequence Diagram	21
4.3.7 Key Objects Involved:	22
4.3.8 Typical Gameplay Flow (Sequence Steps):	22
4.3.9 Purpose of the Sequence Diagram:	23
4.4 DATA DESIGN	23
4.4.1 Transformation of Information Domain to Data Structures	23
4.4.2 Databases or Data Storage Items	24
5. GAME IMPLEMENTATION	24
5.1. UNITY APIs	25
5.2 GAME INTERFACES	26
6. TESTING EVALUATION	31
6.2 INTEGRATION OF GAME COMPONENTS	32

<u>7. PLAY TESTING AND BETA EVALUATION</u>	<u>34</u>
7.1 TESTING SESSIONS CONDUCTED	34
7.2. FEEDBACK THEMES COLLECTED	36
7.3 <i>Outcomes and Improvements</i>	37
<u>8. CONCLUSION AND FUTURE WORK</u>	<u>39</u>
8.1 CONCLUSION	40
8.2 FUTURE WORK	41
8.2.1 <i>Proposed Enhancements</i>	41
<u>9. REFERENCES</u>	<u>43</u>

LIST OF FIGURES

Figure 1 StoryBoard	9
Figure 2 Onion Architecture	14
Figure 3 Activity Diagram	16
Figure 4 Class Diagram	18
Figure 5 Sequence Diagram	22
Figure 6 Key Value Store	23
Figure 7 Start View	26
Figure 8 Map Selection	27
Figure 9 Car Selection	27
Figure 10 Mode Selection	28
Figure 11 Custom Selection	29
Figure 12 Game View	30
Figure 13 Exit Menu	30

LIST OF TABLES

Table 1 Literature Review	2
Table 2 Deliverables	5
Table 3 Development Requirements	7
Table 4 Comparison of Existing Racing game (Turbo Track Rivals).....	8
Table 5 Functional requirements.	10
Table 6 APIs used	25
Table 7 Test cases	31
Table 8 System interactions	33
Table 9 Play testing sessions	35
Table 10 Feedback	36
Table 11 Precautions	38

1. Introduction

"Turbo Track Rivals" is a racing game that provides the player with an exciting and dynamic car racing experience. The vehicles are operated by players via different theme-based tracks and have to negotiate hurdles. The game focuses on realistic vehicle physics, coupled with arcade dynamics to provide the game with excitement. The physics of Unity provides the control of vehicle dynamics such as acceleration, braking, and collisions which provides an immersive experience to the players. The game is comprised of two modes, which include Sprint Mode, a time trial event, and Circuit Mode, a race against other players in three different tracks, including Stadium, Desert Dunes and Snow Track. The different modes and tracks provide a different gameplay to sustain in the racing game.

1.1 Brief

Turbo Track Rivals is a mission-driven racing game that was created with Unity 3D, Csharp, and .NET Framework. It offers a high-speed single-player experience, where the players play high speed vehicles in three specially themed courses: Stadium, Desert Dunes, and Snow Track. The game has some realistic vehicle physics, which is mixed with arcade type of the gameplay whereby there are AI opponents making it a dynamic and interesting game. The project was designed with a modular design approach and based on the Iterative Software Development Life Cycle (SDLC) with tools like Unity 6 (HDRP) to support graphics, Visual Studio Code to support scripting, and Unity Asset Store packages to support track design, sound, and visual effects. The following chapters of this report put a closer examination on various facets of Turbo Track Rivals and the process that made it happen the way it delivered.

To start with, they relate the project to the academic concepts and learning outcomes and demonstrate how the knowledge taught in classrooms was transferred to practice in a hands on manner. Keeping on with the chapters, the author shifts to the inspiration and thinking that led to the development of a racing game, providing an insight into ideas and thoughts that guided the concept. To give ground, other racing games along with their mechanics of the gameplay are also reviewed in the report, and what in them worked good, and what lessons were learnt. This also leads to the comparison of Turbo Track Rivals with other games of the same type, identifying what makes the project stand out and what makes it fit in the current standards. Lastly, the chapters follow the very process of development, including the selection of the methodology, tools, and technical procedures involved to transform the idea of a game into a working experience.

1.2 Relevance to Course Modules

The project is viewed as an implementation of a number of fundamental modules learned during the BS Software Engineering program, and it will combine both basic and specific knowledge. In C#, game mechanics and game behaviours were scripted using concepts of **Object-Oriented Programming (OOP)**, which guaranteed modularity and reuse of that codebase. Software Engineering courses taught the principles that helped in the documentation process, **SDLC modeling**, task allocation and requirements analysis, which offered a systematic structure to the

development process. The game used aspects of Artificial Intelligence in enemy behavior in the forms of pathfinding, patrolling, and dynamic reactions to actions taken by the player. The value of **Human-Computer Interaction (HCI)** was observed in the construction of the user-friendly HUDs, menu systems, and responsive feedback systems to make the experience of the player better. The information acquired in the **Game Development** elective was crucial in building the game mechanics, level design, storyline and the level of interactivity. Lastly, **Data Structures and Algorithms** were used to provide efficiency in data saving, control over enemy states as well as the progression of players and made the performance and play more efficient and smooth.

1.3 Project Background

The concept behind ***Turbo Track Rivals*** was inspired by the excitement and popularity of arcade-style racing games that offer high-speed action combined with creative track designs and engaging gameplay mechanics. The project was initiated with the aim of developing a racing game that not only entertains players but also demonstrates the technical and creative skills acquired during the degree program.

The idea was to create a game that blends realistic vehicle mechanics such as acceleration, braking, and collisions with fun, arcade-style elements and environmental obstacles. The game is set across three distinct tracks: Stadium, Desert Dunes, and Snow Track, each providing a unique atmosphere and gameplay challenges to keep the experience fresh and engaging.

By choosing Unity as the development platform, the project takes full advantage of its physics engine, graphics rendering, and asset integration capabilities. The project also provided an opportunity to explore AI behavior for opponents, modular level design, and smooth control systems. Overall, ***Turbo Track Rivals*** was developed not only to deliver an enjoyable racing game but also to showcase the practical application of game development principles and software engineering techniques.

1.4 Literature Review

Table 1 Literature Review

Game Name	Weakness	Strengths
Forza Horizon 4	Simulation-heavy handling may be too complex or inaccessible for casual players.	<i>Turbo Track Rivals</i> offers simplified, intuitive driving mechanics that are easy to learn but still provide depth and challenge for more skilled players.

Need for Speed Underground	Heavy reliance on custom car modifications can shift focus from core racing mechanics to aesthetics.	<i>Turbo Track Rivals</i> focuses on driving skills and tactical use of power-ups, ensuring that gameplay revolves around the racing experience, rather than overwhelming customization options.
The Crew 1	The game's emphasis on exploration sometimes detracts from the excitement of structured, competitive races.	<i>Turbo Track Rivals</i> offers structured, competitive racing with a focus on fast-paced action, ensuring the adrenaline is constant and players remain engaged in racing rather than exploration.

Alongside insights drawn from commercial games, academic research also played a key role in shaping the technical and design elements of *Turbo Track Rivals*. Topics such as vehicle behavior modeling and physics simulation through Unity's physics engine were studied to create realistic and responsive driving mechanics. Rendering optimization techniques, particularly with the High Definition Render Pipeline (HDRP), were explored to maintain visual quality without compromising performance. Additionally, research into track design and performance tuning provided valuable guidance for achieving smooth gameplay and stable frame rates. Together, these studies informed critical decisions about vehicle dynamics, AI opponent behavior, visual performance, and modular track design, ultimately contributing to a more balanced and immersive racing experience.

1.5 Analysis from Literature Review

From the comparative analysis, it became evident that ***Turbo Track Rivals*** could significantly enhance player engagement by combining thrilling arcade-style racing with technical depth. While many racing games focus heavily on graphics or long-term progression, they often fail to deliver consistent, dynamic gameplay.

Turbo Track Rivals is designed with a blend of features that make each race both exciting and unique. One of the standout aspects is its **AI opponents**, which keep every race fresh and competitive. The opponents respond intelligently to player moves, ensuring that the challenge remains engaging throughout the gameplay. The game also offers **diverse track environments**, including the Stadium, Desert Dunes, and Snow Track each with its own set of obstacles and conditions that push players to adapt their racing style and think strategically. To strike the right

balance, the game combines **realistic vehicle physics** with accessible arcade-style mechanics, allowing both casual players and seasoned racers to enjoy the experience. Together, these elements position *Turbo Track Rivals* as a hybrid racing game that delivers both accessibility and depth, creating a dynamic, skill-driven, and immersive experience for all types of players.

1.6 Methodology and Software Lifecycle for This Project

The software life cycle of this project will be based on a modular approach as this project is a group based project and withing the modular approach the project is divided into parts and those parts are divided into group members who has individually performed their tasks in their respective module.

1.6.1 Rationale behind Selected Methodology (OOA)

The **Modular Object-Oriented Approach (OOA)** was chosen for *Turbo Track Rivals* due to its alignment with Unity and C# principles. Key game elements like Player, Vehicles were implemented as individual classes, ensuring reusability and maintainability.

This methodology was selected because:

- **Supports Component-Based Architecture:** Ideal for Unity, where each game object is treated as a class with modular components.
- **Enhances Collaboration:** Allows team members to work on separate modules simultaneously, speeding up development.
- **Enables Scalability:** New features like vehicles or tracks can be added easily without disrupting existing systems.

This approach ensures a clean, scalable, and maintainable development process.

1.6.2 Rationale behind Selected Methodology (Iterative)

The **Iterative SDLC** was selected for *Turbo Track Rivals* to allow flexibility and continuous refinement throughout development. Game development benefits from constant testing and user feedback, which the iterative model supports. This model enabled:

- **Testing Each Track and Feature:** Each track, vehicle, and game feature were tested individually before final integration.
- **Early Feedback on Gameplay:** Feedback on driving mechanics, AI behavior, and UI was gathered early, allowing for quick adjustments.
- **Continuous Refinement:** AI opponent behavior, track design, and power-up mechanics were tweaked based on testing results.
- **Parallel Development:** Different modules, such as AI, vehicle physics, and UI, were developed in parallel, allowing for efficient progress without delays.

Using iterations allowed for constant improvement in gameplay balance, performance, and overall player experience at every stage of development.

2. Problem Definition

The *Turbo Track Rivals* project aims at addressing the most important challenges in racing games that are usually produced by small teams with minimal resources. Another critical problem is the optimization of performance since the game needs to be running on different devices without compromising the graphics and the frame rate of the game. Finally, modular systems development should be developed, and they must easily be updated and scaled. This modularity means that it can easily add new features (i.e., new vehicles or power-ups) without causing much disruption. *Turbo Track Rivals* aims to provide a casual and difficult racing game experience, which is fun to play but at the same time efficient and technical enough to appeal to the casual and hardcore gamers.

2.1 Problem Statement

To create a game that is interesting and competitive in the racing genre, it is necessary to turn to a balance of excitement, realism of the vehicle behavior, and easy-going performance. The challenges that many racing games, especially those created by smaller teams, encounter include a low variety of tracks to race on, and performance problems between various devices. Also, it might be hard to create the optimal balance of available controls and demanding racing elements, both in time and resource limitations.

The goal of *Turbo Track Rivals* is to overcome these challenges by delivering a racing game with dynamic AI opponents, varied and immersive track designs, and optimized performance across multiple platforms. Given the limited resources and tight development timelines, the project must make efficient design choices while ensuring the game remains engaging, visually appealing, and smooth to play.

2.2 Deliverable and Development Requirements

The deliverables for the Turbo Track Rivals project are divided into software components, documentation artifacts, and assets. Each deliverable plays a critical role in the successful development and evaluation of the final product.

Table 2 Deliverables

Category	Deliverable	Description
	Fully Playable 3D Racing Game	A complete singleplayer 3D Racing game titled Turbo Track Rivals.

Game Development	Three Tracks Based Levels	Stadium, Desert Dunes, and Snow Track
	Opponents	Opponents race against player.
	Player Module	Car physics including handling, accelerating and braking.
	User Interface	Includes HUD (lap count, position, lap time, race time, mini map).
	Game-Saving System	Save/load game progress using local JSON files.
Documentation	Software Requirements Specification (SRS)	Detailed documents outline game requirements and functionalities.
	Feasibility Report	Analysis of technical, financial, and operational feasibility of the project.
	Final Report	Comprehensive report summarizing project outcomes, challenges, and results.
Design Artifacts	Mockups and Storyboard Visuals	Visual designs for levels, UI.
	UML Diagrams	Includes Use Case, Class, and Sequence diagrams for system architecture.
Project Management	Gantt Chart	Timeline and task allocation for tracking project progress and milestones.

The development requirements for Turbo Track Rivals game below:

Table 3 Development Requirements

Category	Requirement	Details
Development Tools	Unity 6.x	Use Unity 6.x with High Definition Render Pipeline (HDRP) for rendering.
	C# Programming	Implement game logic using C# in Visual Studio Code.
	Unity Asset Store	Utilize environments, particle effects, and UI kits from the Unity Asset Store.
	MS Office	For documentation, tracking and project reports
	Draw.io	Modeling the diagrams
Target Platform	Windows 10/11 (64-bit)	Game optimized for Windows 10/11 64-bit systems.
	Processor	Intel Core i5 or equivalent processor.
Minimum System Specs	RAM	4 GB RAM.
	GPU	NVIDIA GTX 460 or higher.
	Storage	8 GB available storage.

2.3 Current System (if applicable to your project)

There is no direct existing system being upgraded in this project; however, *Turbo Track Rivals* takes inspiration from existing Racing games and aims to improve upon their shortcomings. Some examples are referenced in **Table 4**, along with identified weaknesses and the proposed improvements in *Turbo Track Rivals*.

Table 4 Comparison of Existing Racing game (Turbo Track Rivals).

Game Name	Weakness	Strengths
Forza Horizon 4	Simulation-heavy handling may be too complex or inaccessible for casual players.	<i>Turbo Track Rivals</i> offers simplified, intuitive driving mechanics that are easy to learn but still provide depth and challenge for more skilled players.
Need for Speed: Underground	Heavy reliance on custom car modifications can shift focus from core racing mechanics to aesthetics.	<i>Turbo Track Rivals</i> focuses on driving skills and tactical use of power-ups, ensuring that gameplay revolves around the racing experience, rather than overwhelming customization options.
The Crew 1	The game's emphasis on exploration sometimes detracts from the excitement of structured, competitive races.	<i>Turbo Track Rivals</i> offers structured, competitive racing with a focus on fastpaced action, ensuring the adrenaline is constant and players remain engaged in racing rather than exploration.

3. Requirement Analysis

To identify the functional and non-functional requirements of *Turbo Track Rivals*, **storyboarding** was used as the primary technique. This visual method helped map out key gameplay scenarios such as race start and race completion.

By representing game flow and mechanics through visual frames, the team gained a clear understanding of system behavior and user interaction. Storyboarding also enabled better

communication among developers, designers, and testers, ensuring alignment and helping to refine game mechanics before development began.

3.1 Storyboard

Storyboard for **Turbo Track Rivals** below.

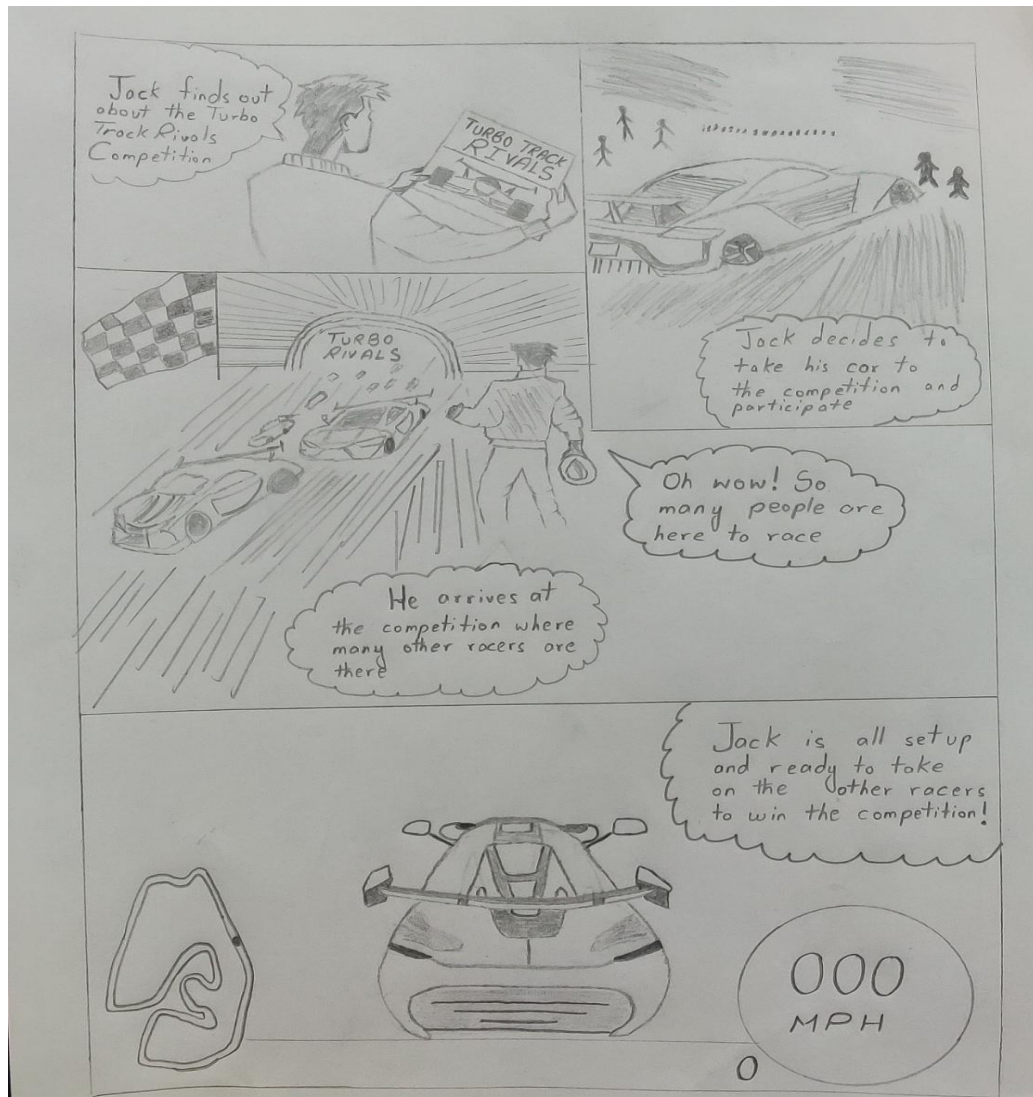


Figure 1 StoryBoard

The storyboard of *Turbo Track Rivals* centers on fast-paced competition where the player must outsmart AI opponents and cross the finish line first to win. The game features **two modes: Time Trial**, focused on racing against the clock to achieve the best lap times, and **Sprint Race**, where players directly compete with adaptive AI rivals in high-stakes races.

Players can race across **three distinct maps** the **Stadium Track** for pure speed, the **Desert Dunes** with sharp turns and sandy obstacles, and the **Snow Track** with slippery surfaces and limited visibility. Each environment offers unique challenges that encourage different racing strategies.

With its blend of competitive AI, varied game modes, and diverse tracks, *Turbo Track Rivals* delivers an engaging and replayable racing experience.

3.1.1 Storyboard description

Jack discovers the exciting *Turbo Track Rivals* competition through an advertisement and immediately decides he wants to participate. Motivated by the thrill of the challenge, he grabs his keys, heads to the event, and brings his car, ready to race. Upon arriving at the grand competition stadium, Jack is amazed by the sheer scale of the event—the roaring engines, cheering spectators, and flashing lights create an electric atmosphere. He realizes this is more than just a race; it's a festival of speed and competition.

After parking his car in the pit area, Jack takes a moment to observe the other racers. Each one is tuning their vehicles, adjusting tires, and running last-minute checks with intense focus. The sight fuels his determination, but also reminds him of the fierce level of skill he's about to face. The tension builds as the announcer's voice booms across the stadium, introducing the racers and hyping up the crowd.

When the time finally comes, Jack walks toward the starting line. With the weight of the moment pressing on him, he straps on his helmet, grips the wheel tightly, and revs his engine. The AI opponents line up beside him, their cars gleaming under the stadium lights, engines rumbling like wild beasts ready to charge. Jack takes a deep breath, focusing his mind and steadying his nerves. The countdown begins, the crowd erupts in excitement, and Jack prepares to unleash everything he has in the ultimate test of speed, strategy, and skill the legendary *Turbo Track Rivals*.

3.2 Functional Requirements

This section outlines the functional requirements of *Turbo Track Rivals* using a natural language approach. The requirements are categorized based on gameplay mechanics, user interactions, and core system processes. These requirements aim to ensure a smooth and immersive racing experience from vehicle control and AI behavior to UI navigation and game mode selection.

The game is designed to handle player inputs, racing logic, error conditions (e.g., invalid selections), and state transitions effectively. Each requirement supports the goal of delivering an engaging, responsive, and enjoyable user experience.

Identifier	Requirement ID	Title	Requirement	Source	Rationale	Dependencies	Priority

High	
FR- ² , FR -3	
To ensure a realistic and engaging driving experience with smooth car controls.	
Game Design Document	
The player shall be able to control the car's acceleration and braking using the keyboard. The car should accelerate smoothly and decelerate realistically based on player input.	
Car Acceleration and Braking	
FR-1	
FR-1	

High	High	Low
FR-1, FR-3	FR- ¹ , FR -2	FR-1
To give the player a sense of control over the car while enhancing the racing experience with varying responses to steering.	To add realism to the racing game, with impacts when collisions happen, requiring the player to be cautious and strategic.	To enhance the racing atmosphere and immersion by creating dynamic audio cues based on the player's actions.
Game Design Document	Game Design Document	Game Design Document
The player shall be able to steer the car left and right with smooth, responsive controls. Car handling shall be influenced by speed.	The system shall detect collisions with other cars, track barriers, and objects. Upon collision, the car's speed shall be affected based on the impact.	The game shall play background music and realistic sound effects, such as car engines, tire screeches, and collisions.
Car Handling and Turning	Collision System	Background Music and Sound Effects
FR-2	FR-3	FR-4
FR-2	FR-3	FR-4

Medium	High
FR-1, FR-2	FR-1, FR-2
To ensure the player has control over their session, allowing them to pause the game in case of interruptions or to restart the race if necessary.	This behavior system ensures that every race feels challenging, enhancing player engagement and replay value.
Game Design Document	Game Design Document
The system shall provide a pause menu with options to resume, restart the race, change settings, or quit the game. If the game is paused, the race should stop immediately.	The system shall implement opponents with dynamic racing behavior.
Pause and Restart Options	Opponent Car behaviour
FR-5	FR-6
FR-5	FR-6

Table 5 Functional requirements.

3.3 Non-Functional Requirements

This part describes the non-functional (quality) requirements of *Turbo Track Rivals*. These requirements specify the way in which the system is to act as opposed to what it will act. These are meant to make sure the game performs as per the expectations, requirement of the users and also to make sure the game is maintainable. The requirements are specific, measurable and verifiable to serve as a guideline in the implementation and testing.

Turbo Track Rivals is a well-planned game with various non-functional requirements being directed towards the establishment of a smooth, reliable, and approachable experience to the players. Having said that, the game is optimized to run with at least **30 FPS** with the mid-range system that has a **GTX 1050** graphics card, **8 GB of RAM**, and Intel **i5 6 th generation** processor. This makes the game experience smooth and comfortable without experiencing any lag or frame drops. To be easy to navigate, the interface is user-friendly and intuitive, and the player can navigate menus, switch to various modes of the game, and set up the settings without learning too much or being confused by the needless complexity.

Stability is also taken into consideration and the game is expected to have 99.9% uptime on average gameplay conditions so that few disruptions can be brought about by crashes. To enhance **reliability**, the system will save progress automatically after every race to ensure the players are not lost when there are unexpected shutdowns or errors. Lastly to ensure the game is accessible, the game has support of keyboard controls so that players do not need any extra or special devices to enjoy the game.

4. Design and Architecture

The following parts of Software Design Description (SDD) report should be included in this chapter.

4.1 System Architecture

The system Architecture is followed as Onion Architecture, which is most common in Game development, it is modularized into key components:

4.1.1 Onion Architecture (Turbo Track Rivals)

The Onion Architecture in *Turbo Track Rivals* organizes the game into clear-cut modular layers, which guarantee the clear **separation of concerns**, enhanced **maintainability**, and **scalability** over time. The domain layer is at the core and it contains the game logic which is the basic rules of the game, vehicle dynamics, scoring, and AI opponent behaviour. Enclosed in this is the application layer that handles use cases such as race initialize, lap tracking, saving progress and interaction among the game components.

The infrastructure layer is the next layer which addresses technical issues like data storage, input/output, rendering pipelines and connecting with Unity physics engine. The presentation layer is the last layer and here the player can interact with the game, using easy to use intuitive menus, **HUDs**, **audio effects**, and narrative-driven aspects to make the game more immersive.

Through such layered structure of the game, any modifications to a specific aspect of the game (e.g. a new vehicle physics or the addition of a new UI element) can be performed with the least amount of effect on other aspects of the game, minimizing the likelihood of causing bugs. It is also simpler to add new features to the game in the future, be it by adding new songs, improving the AI behavior, or including the multiplayer experience. Finally, the Onion Architecture helps to make *Turbo Track Rivals* modular, flexible, and durable, to accommodate the existing requirements of the game and expand in the future.

In Onion Architecture within *Turbo Track Rivals*, the game is structured into distinct modular layers, which ensure the distinct separation of concerns, higher maintainability, and scale. The domain layer is at the central point and it holds the game logic which is the fundamental rules of the game, vehicle dynamics, scoring and the AI opponent behaviour. Within this is the application layer which, among other use cases, deals with the race initialize, lap tracking, saving progress and interaction between the game elements.

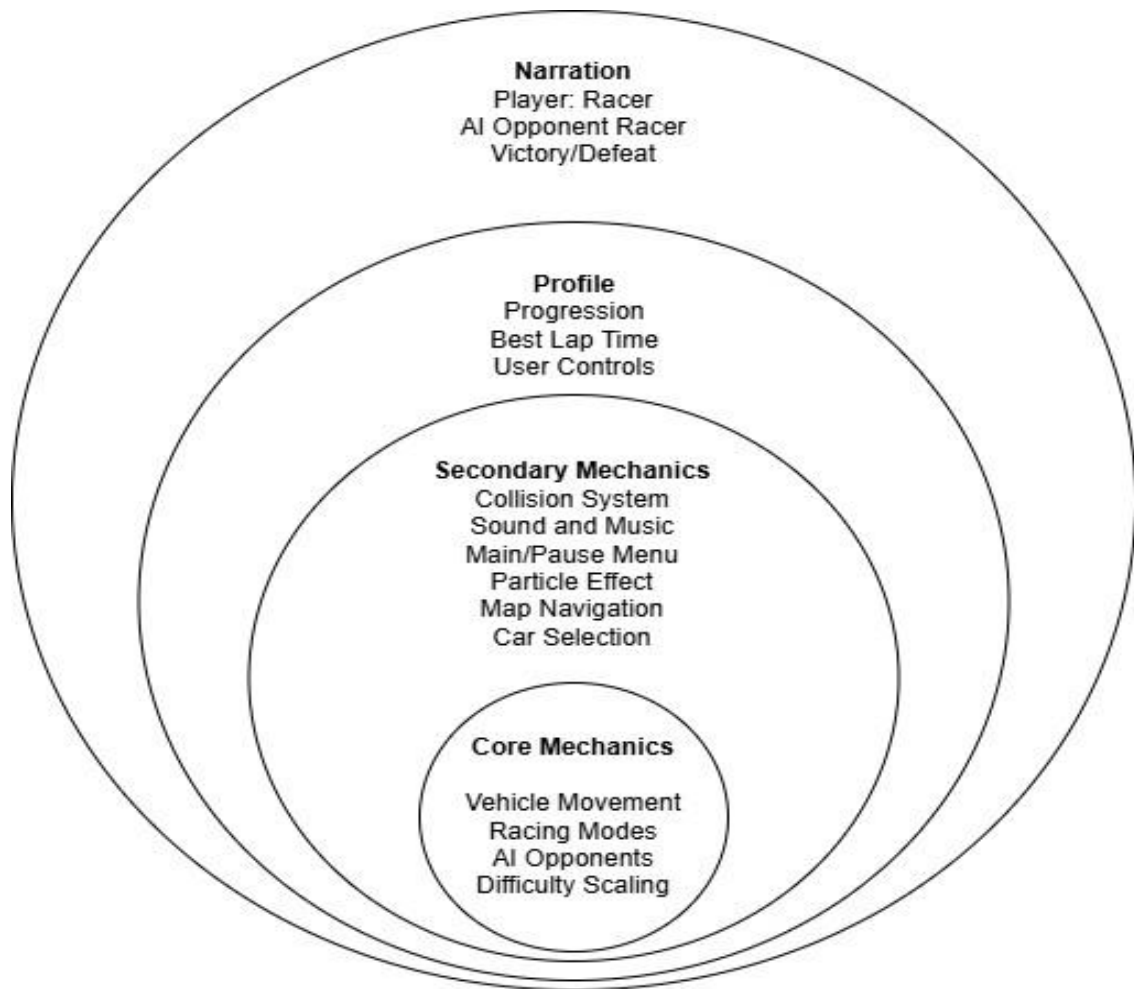


Figure 2 Onion Architecture

4.1.2 Core Mechanics (Center Layer)

This layer contains the foundational systems responsible for the main gameplay logic:

- **Vehicle Control:** Manages player inputs for acceleration, braking, steering, and drifting.
- **Lap and Timer System:** Tracks lap counts, timing, and race completion logic.
- **Opponent AI:** Handles enemy vehicles' racing logic using waypoints and adaptive behavior.

4.1.3 Secondary Mechanics (Second Layer)

These systems support and enhance the core gameplay experience:

- **Collision Detection & Response:** Handles car-to-car and car-to-obstacle interactions.
- **Sound and Music System:** Plays engine sounds, background tracks, and power-up effects.
- **Visual Effects:** Renders effects like sparks and finish line animations.
- **Track Navigation Aids:** Provides mini maps, checkpoints, and directional arrows.

- **Main Menu & Pause System:** Offers access to settings, game mode selection, and pause functionality.

4.1.4 Profile Layer (Third Layer)

This layer handles data persistence and game customization features:

- **Save & Load System:** Allows saving player progress and game states.
- **Player Profile Management:** Stores unlocked vehicles, best times, and achievements.
- **Track Unlock Logic:** Controls access to new tracks based on player progress.
- **Settings Toggle:** Lets players adjust audio, controls, and display preferences.

4.1.5 Narration Layer (Outer Layer)

Provides context, theme, and visual identity to the gameplay:

- **Racer Identity:** Represents the player's customizable vehicle or character.
- **Opponent Backstories/Designs:** Defines the look and racing style of AI opponents.
- **Race Themes and Descriptions:** Explains the environment and objective of each track (e.g., Stadium, Desert Dunes, Snow Track).
- **Intro/Outro Sequences:** Cinematic moments or visual intros before or after races to enhance immersion.

4.2 Process Flow/Representation

The **process flow** provides a detailed overview of the sequential progression of activities within *Turbo Track Rivals*, visually represented in the accompanying flowchart. It illustrates how the game transitions from one stage to another, highlighting key interactions and decision points throughout the player's journey. Starting from the launch of the game, the flowchart traces the progression through main menus, map and car selection, mode customization, and finally entering the race itself. It also depicts system responses, such as real-time updates to the user interface, AI behavior adjustments, lap counting, and race completion sequences. By presenting the game's operations in a structured and visual format, the process flow helps clarify the relationships between different modules, ensures smooth gameplay transitions, and serves as a guide for developers and testers to understand the overall system logic and interaction patterns.

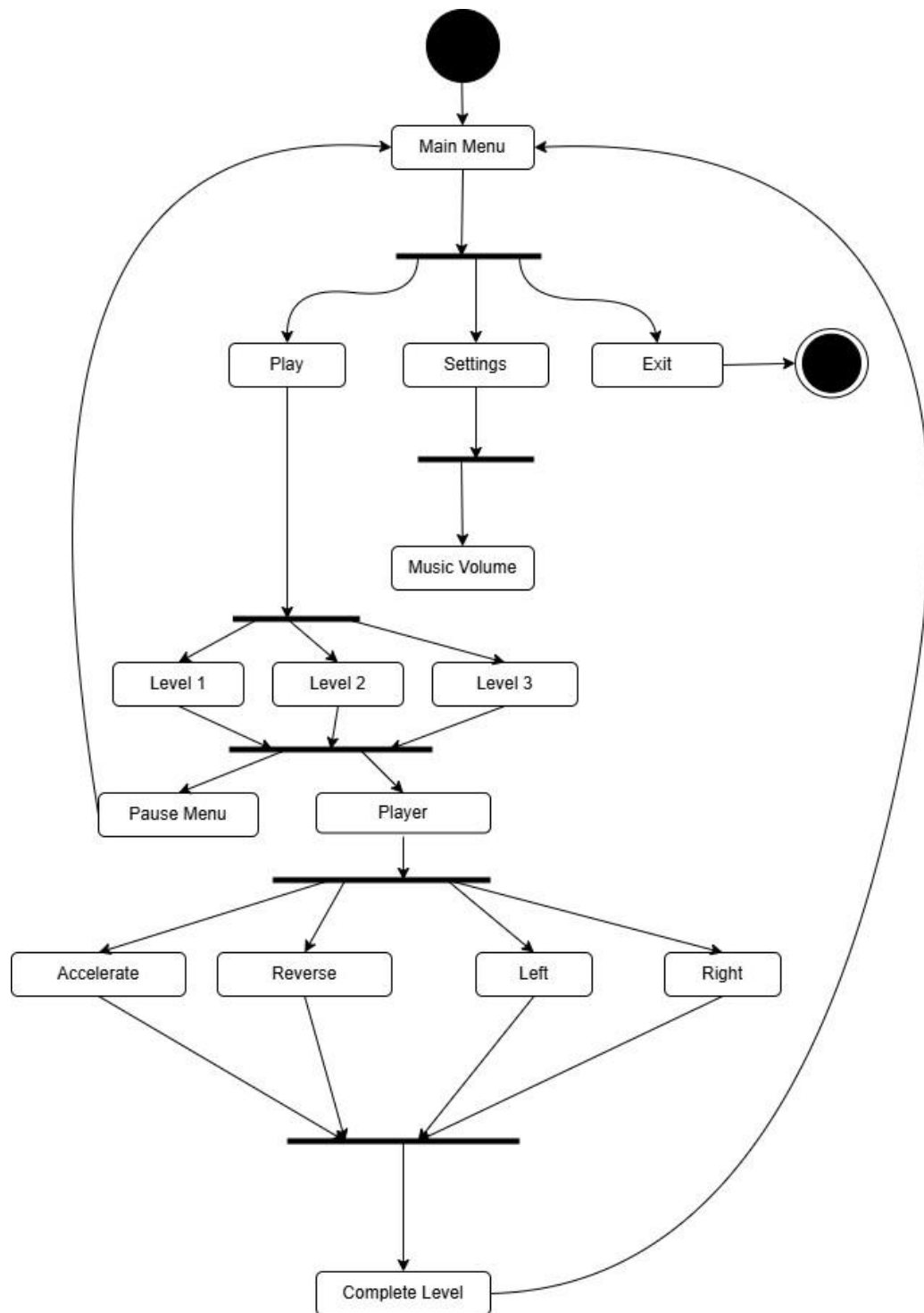


Figure 3 Activity Diagram

4.2.1. Description Activity diagram

An **Activity Diagram** visually represents the flow of gameplay actions and system activities in *Turbo Track Rivals*. It shows the sequence of tasks a player performs from launching the game to completing a race, including system responses. This diagram helps in understanding user interaction, control flow, and decision points within the game.

The diagram begins with the **Start** node and follows a structured flow through key game phases:

- **Main Menu Navigation:** Players choose between Sprint Mode or Circuit Mode, access settings, or exit the game.
- **Track and Car Selection:** Players select their track (e.g., Stadium, Desert Dunes, Snow Track) and vehicle before beginning the race.
- **Race Initialization:** The game loads assets, positions players and AI cars, and starts the countdown.
- **Race Execution:** During the race, the player interacts with:
 - **Vehicle Controls** (accelerate, brake, steer).
 - **Lap Tracking**
 - **Opponent Car Interactions**
- **Win/Loss Condition Check:** After completing all laps or reaching the goal, the game checks whether the player has won or lost.
- **Post-Race Options:** Display results, allow replay, return to main menu, or exit the game.

Decision nodes are used throughout the diagram to represent choices like:

- Game mode selection
- Retry or quit after the race
- Track selection based on unlock status

The **Activity Diagram** ensures a clear visualization of how players engage with the game and how the game system responds at each step. It is especially helpful during the design phase for identifying flow bottlenecks or areas needing additional features.

4.3 Design Models [along with descriptions]

The **Turbo Track Rivals (3D Racing Game)** includes Class representation and sequence for that class, which are the following below.

4.3.1 Class Diagram

The class diagram acts as a blueprint for Turbo Track Rivals, laying out the main objects in the game and showing how they connect with one another. It highlights the attributes, methods, and relationships between different classes, giving developers a clear picture of how the game is structured behind the scenes. The object-oriented design is laid out visually, making it easier to have a grasp of the code structure, the way various components of the system interrelate, and how it will be possible to design future changes or additions to it. The following is the necessary Class Diagram:

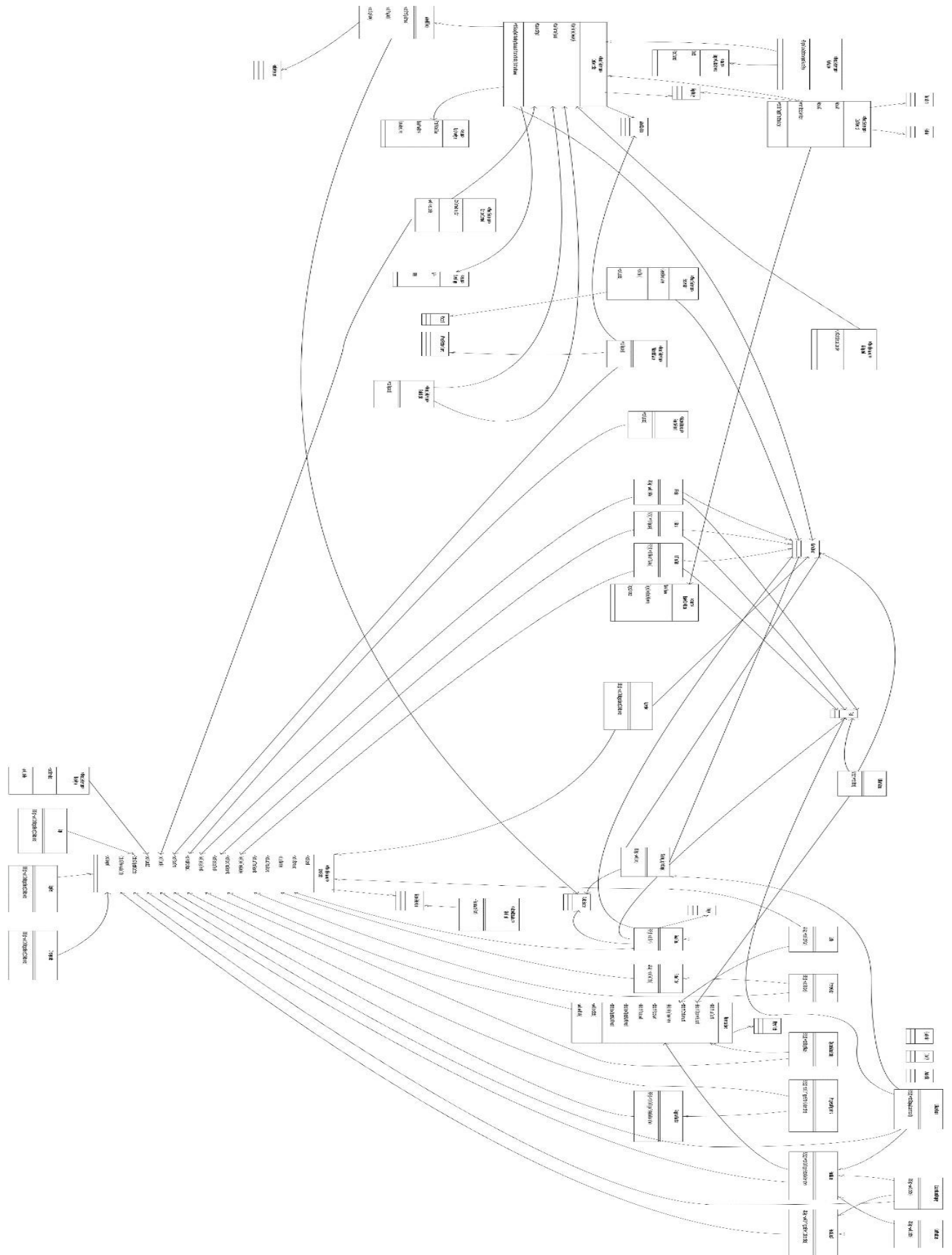


Figure 4 Class Diagram

4.3.2 Class Diagram Description (Turbo Track Rivals)

The Class Diagram provides a structural overview of the core objects in *Turbo Track Rivals*, showcasing how different classes relate to each other in terms of attributes, methods, and associations. It reflects the object-oriented design of the game, helping developers clearly understand the overall architecture and code organization. Core entities such as **Player**, **Vehicle**, **AI Opponent**, **Track**, and **Race Manager** are represented, each encapsulating specific responsibilities to maintain modularity and reusability.

For instance, the **Player** class may contain attributes such as player name, progress, and statistics, along with methods to control inputs and interact with the game world. The **Vehicle** class defines properties like speed, acceleration, and handling, as well as behaviors related to physics and performance tuning. The **AI Opponent** class extends vehicle behavior with additional logic for pathfinding, decision-making, and adaptive difficulty. Similarly, the **Track** class manages layouts, obstacles, and environmental conditions across different maps, while the **Race Manager** coordinates the flow of the game, including lap tracking, race results, and progress saving.

By mapping these relationships, the Class Diagram not only illustrates how classes collaborate but also serves as a blueprint for implementation, debugging, and future extensions. It enables developers to identify dependencies, maintain clean code separation, and ensure that new features such as multiplayer or additional tracks can be integrated smoothly without disrupting existing functionality.

4.3.3 Key Classes and Their Roles:

1. GameManager

- **Role:** Central controller that manages game flow.
- **Attributes:** currentMode, currentTrack, gameState
- **Methods:** StartGame(), EndGame(), PauseGame(), LoadScene()

2. PlayerCar

- **Role:** Represents the user-controlled vehicle.
- **Attributes:** speed, acceleration, brakeForce, currentLap, powerUps
- **Methods:** Accelerate(), Brake(), Steer(), UsePowerUp(), UpdateLap()

3. OpponentCar

- **Role:** AI-controlled racing vehicles.
- **Attributes:** speed, aiLevel, pathPoints, currentLap
- **Methods:** FollowPath(), AvoidCollision(), ReactToPlayer()

4. TrackManager

- **Role:** Manages racing track information and checkpoint validation.
- **Attributes:** trackName, totalLaps, checkpoints[], theme

- **Methods:** LoadTrack(), ValidateLap(), RestartTrack()

5. PowerUpManager

- **Role:** Controls spawning, pickup, and effect application of power-ups.
- **Attributes:** activePowerUps, spawnLocations
- **Methods:** SpawnPowerUp(), ApplyPowerUp(), RemovePowerUp()

6. UIManager

- **Role:** Handles all on-screen UI elements like timers, positions, menus, etc.
- **Attributes:** raceTimer, lapDisplay, playerPosition, menus[]
- **Methods:** UpdateUI(), ShowResults(), TogglePauseMenu()

7. SoundManager

- **Role:** Manages game audio including engine sounds, effects, and music.
- **Attributes:** engineSound, bgMusic, sfx[]
- **Methods:** PlaySound(), MuteAll(), ChangeMusicTrack()

8. PlayerProfile

- **Role:** Stores player-specific data like progress, best times, and settings.
- **Attributes:** unlockedTracks, vehicleSkins, highScores
- **Methods:** SaveProfile(), LoadProfile(), ResetData()

4.3.4 Relationships and Interactions:

- GameManager **aggregates** TrackManager, UIManager, SoundManager, and PowerUpManager.
- PlayerCar and OpponentCar **inherit** from a base class Car (optional), which contains shared attributes like speed, acceleration, and methods like Move ().
- PowerUpManager interacts with both PlayerCar and OpponentCar to apply effects.
- UIManager communicates with GameManager, TrackManager, and PlayerProfile to fetch and display game status.

4.3.5 Purpose of the Class Diagram:

- Clarifies the **architecture** for all core game objects.
- Supports **modular development** by identifying responsibilities clearly.
- Helps in debugging and extending features (e.g., adding new game modes or AI types).
- Encourages **code reuse** through inheritance and component-based design.

4.3.6 Sequence Diagram

The Sequence Diagram is an interaction diagram of different elements of a game. This flow chart illustrates in detail the sequence of interaction that occurs when a race is going on in Turbo Track Rivals. It graphically illustrates the manner in which various objects and systems interact with one another overtime and indicate the sequence in which things happen. It is a good way to see the dynamic behavior of the system during the play as it maps out the interactions of the game in response to the actions of the player and the interactions between different parts of the game, exchanging information.

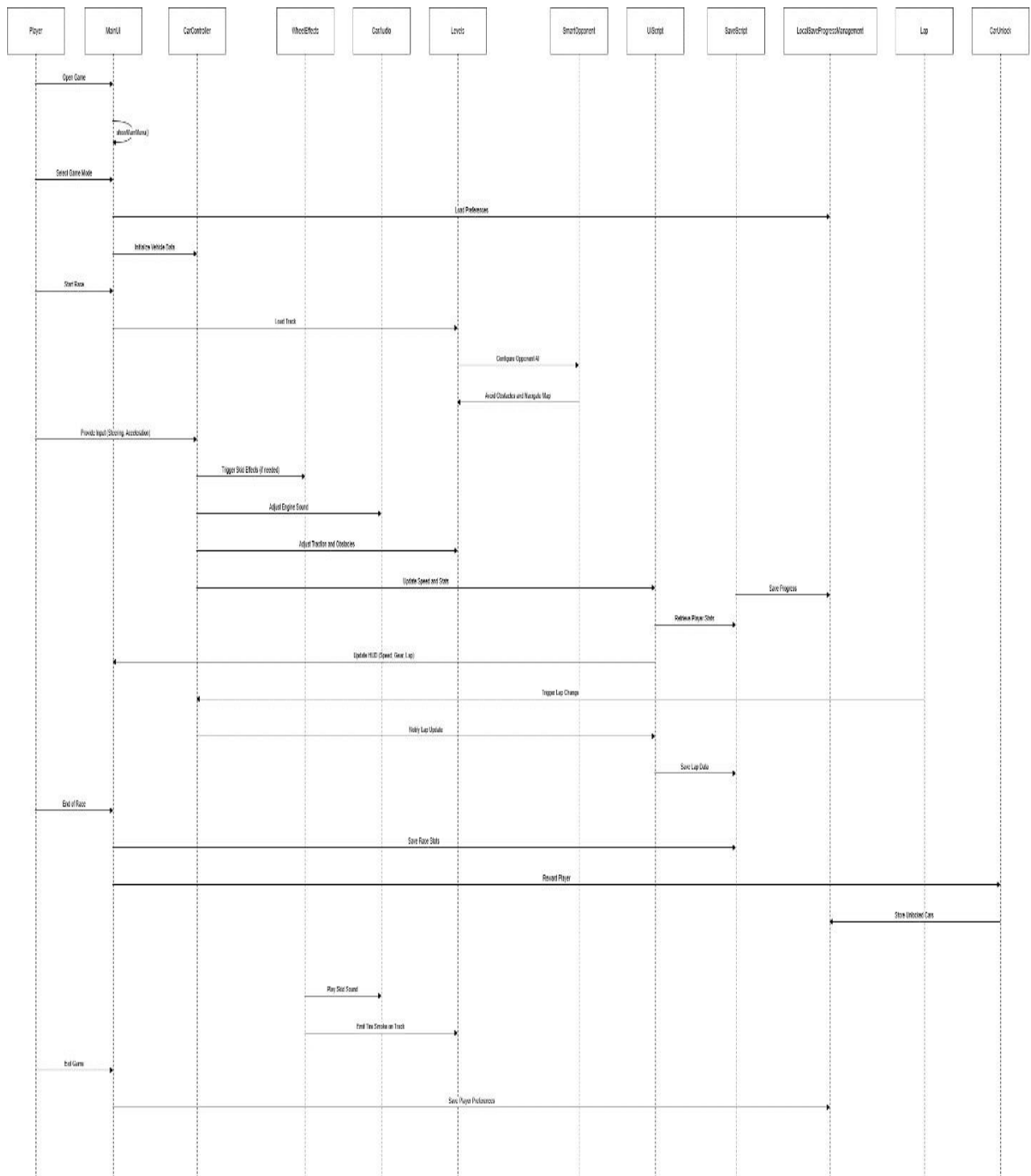


Figure 5 Sequence Diagram

The **Sequence Diagram** illustrates the flow of interactions between different objects and systems during a typical race scenario in *Turbo Track Rivals*. It outlines how the system responds to player input and how components communicate with one another in a step-by step, time-ordered manner.

This visualization helps understand the **runtime behavior** of the game during gameplay, specifically during a **race start-to-finish flow**.

4.3.7 Key Objects Involved:

1. Player (User)
2. GameManager
3. UIManager
4. TrackManager
5. PlayerCar
6. OpponentCar
7. SoundManager

4.3.8 Typical Gameplay Flow (Sequence Steps):

Game Flow Sequence

When the player starts the game, the **GameManager** is first contacted to load the **main menu**. From there, the player chooses a game mode and track, which prompts the **GameManager** to call on the **TrackManager** to load the selected environment. At the same time, the **UIManager** updates the interface to reflect the player's chosen options.

Once the setup is complete, the **GameManager** begins the countdown sequence, during which both the **PlayerCar** and the **OpponentCar** are spawned and made ready for the race. In parallel, the **SoundManager** plays the background music along with the countdown effects.

As soon as the race begins, the player starts providing inputs such as steering and acceleration. These inputs are passed to the **PlayerCar**, which responds by moving accordingly and continuously sending lap progress and timing information back to the **UIManager**.

To maintain fairness, the **TrackManager** keeps track of checkpoint validation for both the player and AI opponents, ensuring laps are completed correctly. When the race ends, the **GameManager** delivers the final results to the **UIManager**, which then displays the outcome screen with the win/loss status and post-race statistics. Simultaneously, the **SoundManager** transitions the audio to either victory music or a failure theme, bringing the experience to a close.

4.3.9 Purpose of the Sequence Diagram:

The usefulness of a sequence diagram is that it can easily show the sequence of interaction among various objects during the race moment, thus being able to comprehend how the different components interact in real time. Therefore, it also identifies dependencies between essential game modules like the **GameManager**, **TrackManager**, **UIManager**, etc. by mapping out such interactions. This visibility is particularly important when developing and testing because it assists developers to determine possible problems, debug the behavior of the system and how to know the system will integrate smoothly. Moreover, the sequence diagrams make sure that all details of the game like the input of the players, the rules of the game, visuals and the auditory response are timely aligned. This synchronization does not only enable a smooth gameplay, but also helps to improve the user experience in general, reducing errors and providing the system with stable responses.

4.4 Data Design

Turbo Track Rivals data design is aimed at systematizing the most important data of the game into standard formats that can be easily updated and managed. The fundamental data that comprises player profiles, cars, tracks, race results, and power-ups are in the form of clear data structures to facilitate a smooth gameplay and progression. This design will be used to make sure that data can be stored, accessed and edited in an efficient manner either due to in-game logic, user interface updates or long term player development. It is scalable and modular, so it can be used with such formats as ScriptableObjects, simple databases, or JSON files.

4.4.1 Transformation of Information Domain to Data Structures

The information domain for *Turbo Track Rivals* is transformed into structured data formats that represent core gameplay components such as player profiles, race results, car selection, and track progress. These data structures are designed to support efficient game logic execution, real-time updates, and persistent player progression. The hierarchy follows a modular and scalable approach, suited for JSON files, ScriptableObjects, or document-based databases.

PlayerPrefs Key-Value Store

PlayerPrefs
MyCredits : int
PlayName : string
WonRaces : int
LostRaces : int
MyCar : int
Car1 : int (0 = not owned, 1 = owned)
Car2 : int (0 = not owned, 1 = owned)

Figure 6 Key Value Store

4.4.2 Databases or Data Storage Items

The database of Turbo Track Rivals is structured in such a manner that the information about players, game progress and settings can be stored in a well organized and easy to retrieve form. **Firebase** or **MongoDB** is a database that is highly appropriate because it is flexible and can handle dynamic data models. **NoSQL** Games allow the storage of core gameplay data such as player statistics, track progression and user interface preferences in collections and documents and therefore it is easy to update gameplay and access this information when required. This method of storage makes it possible to synchronize in real time, make queries using indexes and integrate well with the entire game architecture.

4.4.3 Database: NoSQL Database (e.g., Firebase, MongoDB)

In the database design, the main collections would include **playerData**, **levelProgress**, and **userInterface**. Each player's data is stored as a document inside the **playerData** collection, containing fields such as *bestLap*, *lapTime*, *raceTime*, *lapCount*, and *position*. The **levelProgress** collection would store sub-documents or arrays for *level1*, *level2*, and *level3*, representing the player's progress through different stages of the game. Similarly, the **userInterface** collection could include a document containing *audioSettings*, with fields such as *volume* to manage sound preferences. To improve efficiency, indexes may be applied to frequently accessed fields such as *position* or specific **levelProgress** fields, optimizing the loading and saving of gameplay progress.

4.4.4 Files

If the system is kept lightweight, all this information (from **playerData**, **levelProgress**, and **userInterface**) could be serialized and stored in a JSON file locally. This would be useful for local save/load functionality, as shown by the Local Save - Progress Management node connected to multiple data points.

4.4.5 In-Memory Storage

Transient data such as the current *lapTime*, *raceTime*, and *position* during gameplay would be stored in memory for fast access and real-time performance. Once the race ends or progress needs to be saved, the data would be written to persistent storage like NoSQL or JSON.

5. Game Implementation

This chapter details the core systems implemented in *Turbo Track Rivals*, a single-player mission-based racing game. These systems include vehicle control mechanics, AI opponent behavior, track-based obstacle management, power-up interactions, and environmental effects. The game is developed using Unity and scripted in C#, leveraging Unity's built-in features and APIs for key systems including physics, input, audio, visual effects, animation, and scene management.

5.1. Unity APIs

Turbo Track Rivals makes extensive use of Unity's native API modules. These APIs support essential systems such as vehicle input, sound effects, AI-driven opponents, user interface, and scene transitions across different tracks.

Table 5.1: APIs Used

Table 6 APIs used

API Module	Description and Usage
UnityEngine	Base Unity framework providing access to core components like MonoBehaviour, GameObject, and game loop behavior.
UnityEngine.InputSystem	Captures vehicle controls such as acceleration, braking, steering,
UnityEngine.Audio	Manages in-game sound effects such as engine sounds, drifting, collision impacts, and ambient track audio.
UnityEngine.Animations / Animator	Handles environmental animations such as moving barriers, checkpoint indicators, and power-up activations.
UnityEngine.Physics	Manages collisions, rigidbody dynamics for realistic vehicle physics, and triggers for track-based obstacles and power-ups.
UnityEngine.SceneManagement	Controls transitions between the main menu, different track scenes (Stadium, Desert Dunes, Snow Track), and game modes.
UnityEngine.Events	Supports custom triggers for events like lap completion, race start/end, checkpoint crossing, and power-up collection.
UnityEngine.AI	Powers AI opponent vehicles using NavMesh for track navigation and pathfinding during Circuit Mode.
UnityEngine.UI	Renders in-game UI such as speedometer, timer, lap count, minimap, and game status indicators.

5.2 Game interfaces

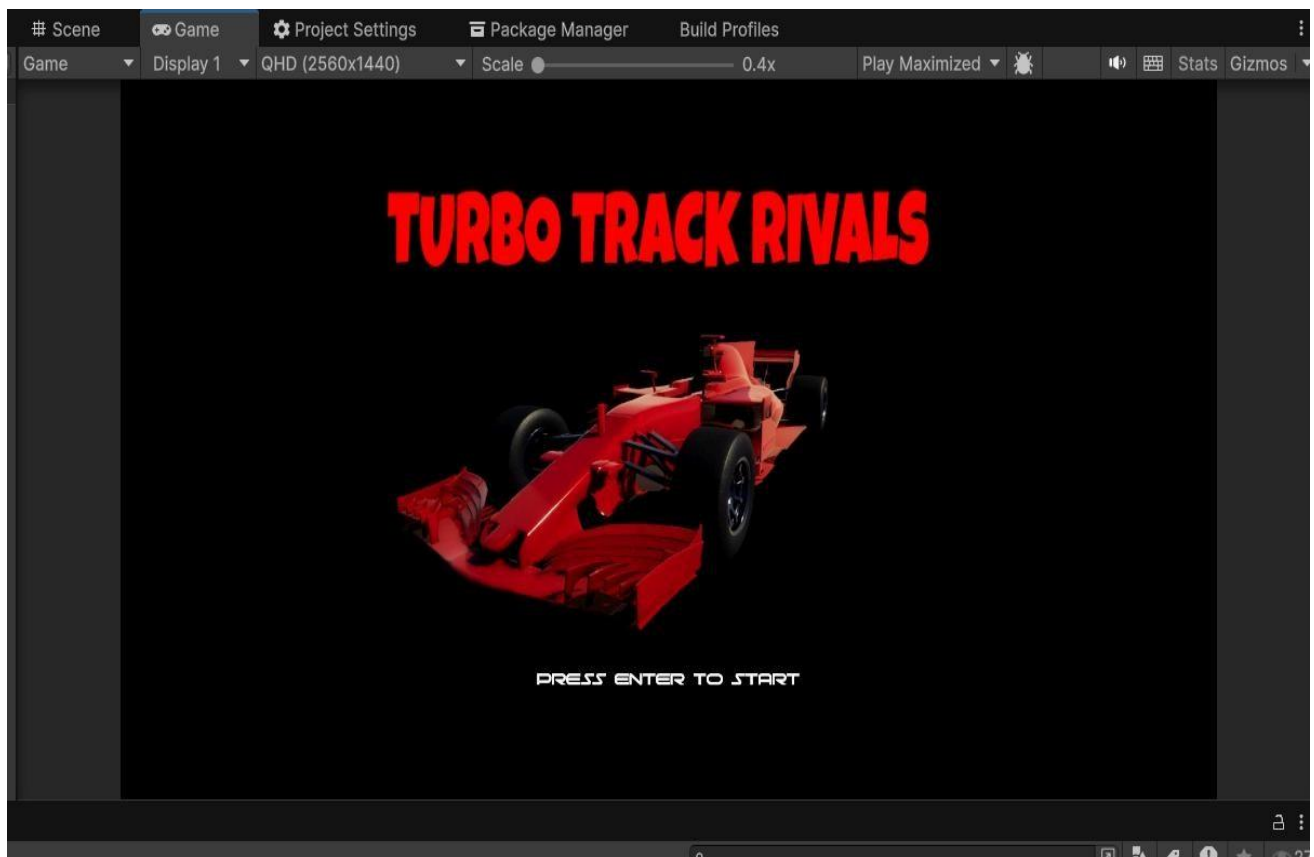


Figure 7 Start View

The Start View is the access point of the *Turbo Track Rivals*, which is the default display on opening the executable. The game can be started and the next scene can be easily switched as soon as a player presses the Enter key. It is not only the tone created by photos and the mood but also makes it easy to access the game, since it can be started as an executable or in a packaged version. The Start View can be also improved in the next versions by adding features such as **New Game**, **Continue**, **Settings** and **Exit** to offer increased flexibility to players.

The key assigned to switch from the menu screen to the main menu screen is the enter key, by pressing the enter key it takes the players to the next scene which should be the main menu, there is a seamless scene switching used which provides the best user experience while switching from different scenes throughout the whole game, the goal is to provide a good user experience and to make the players enjoy the game without any buffers and long loading times.

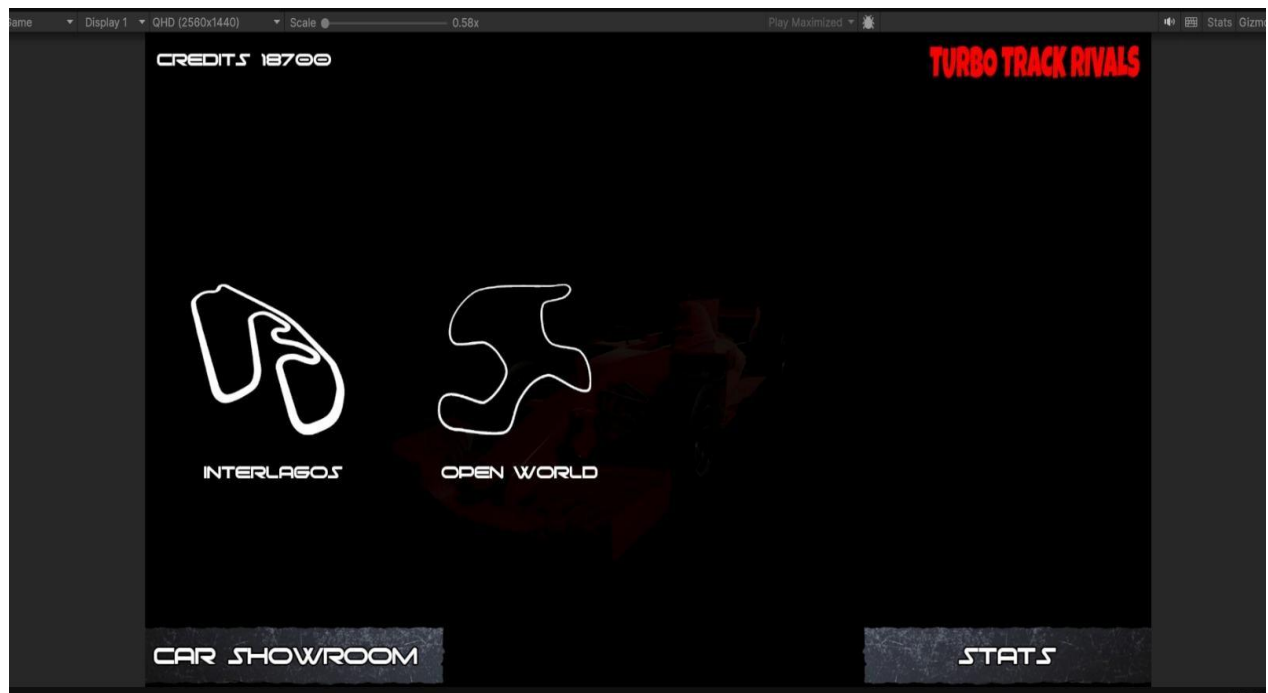


Figure 8 Map Selection

Upon clicking on the **Main Menu**, it now results to the **Map Selection** screen, which allows players to select one of the racing environments and game modes. Every map like the Stadium, Desert Dunes or Snow Track has its own obstacles, terrain and atmosphere, which prompts the players to change their race style. The mode selected, be it **Time Trial** or **Sprint Race**, then advances to its own scene, allowing an easy transition to the game play. The fact that it gives players the ability to play the game in their preferred style is also a benefit in that the game is made to be more replayable since one can choose different maps and play modes to ensure that each race is unique and interesting.

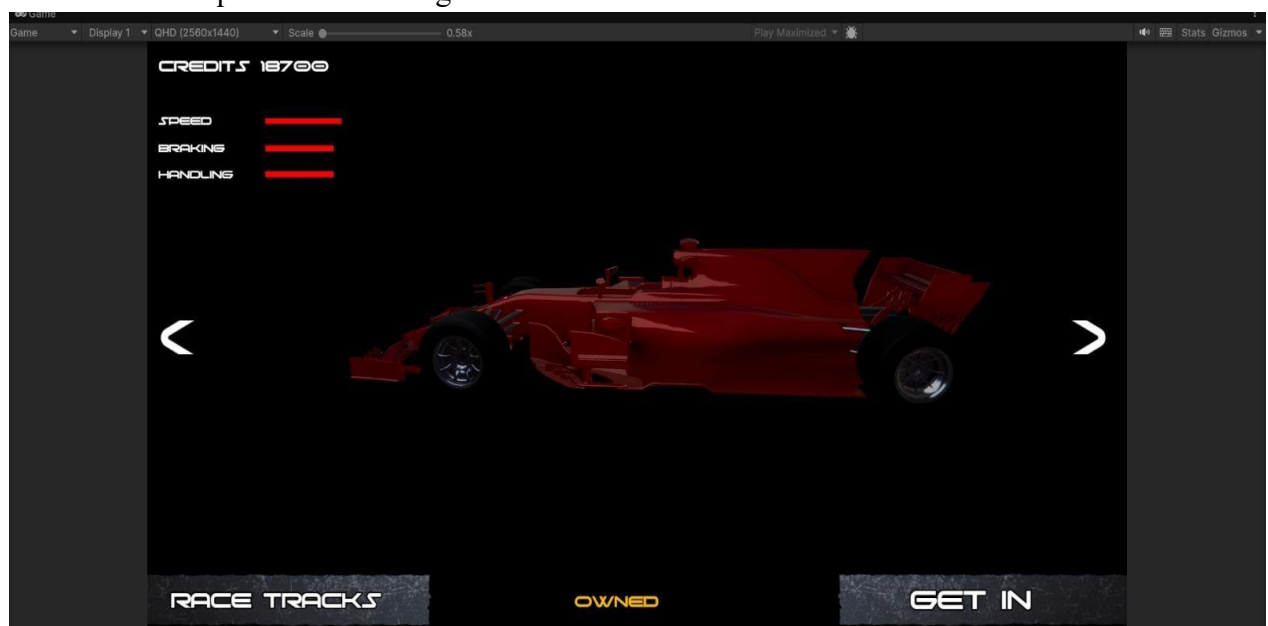


Figure 9 Car Selection

The **Car Selection** screen takes them to an interactive showroom where they are allowed to shop around a variety of cars, each with very distinct looks and car performance features like speed, handling and acceleration. In order to make the experience further individualized, the players can modify the car they have chosen by changing the color, which will enable them to feel personal and possessing the car. This functionality not only increases the level of immersion but also provides the players with a choice of how they want their vehicle to be. Through the use of choice and customization, the **Car Selection** screen creates hype prior to the start of the race, and the transition to the gameplay process becomes more exciting and satisfying.

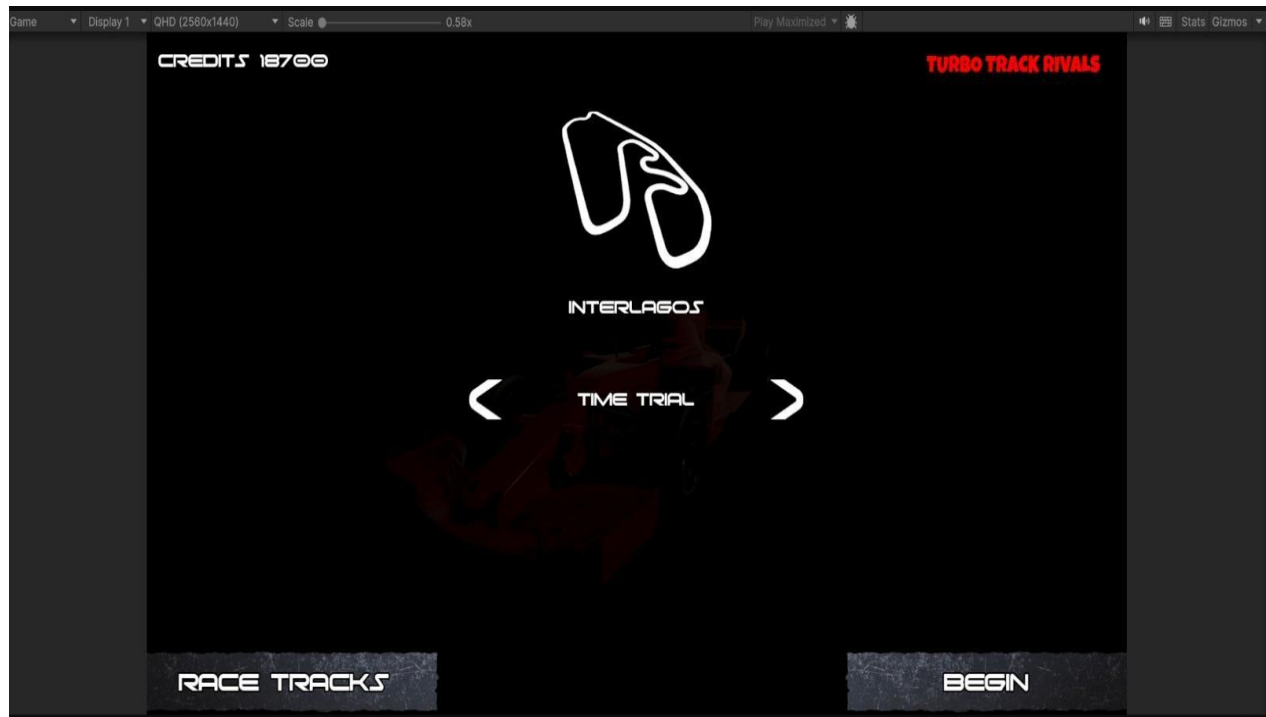


Figure 10 Mode Selection

Mode Selection screen allows players to switch between various gameplay modes which each map offers, like **Lap Race** or **Time Trial**, which will be loaded into their own unique scene. **Lap Race** is all about racing with the AI, where the player has to beat the other contenders to the finish line, whereas **Time Trial** is all about accuracy and speed, with the player racing the clock in order to achieve the best lap times. The flexibility then provides the player with the choice to customize their experience either to strategic racing that is competitive or individual skill-based challenges. **Selecting mode** and using various maps brings a lot of replayability to the game which does not make two races identical.

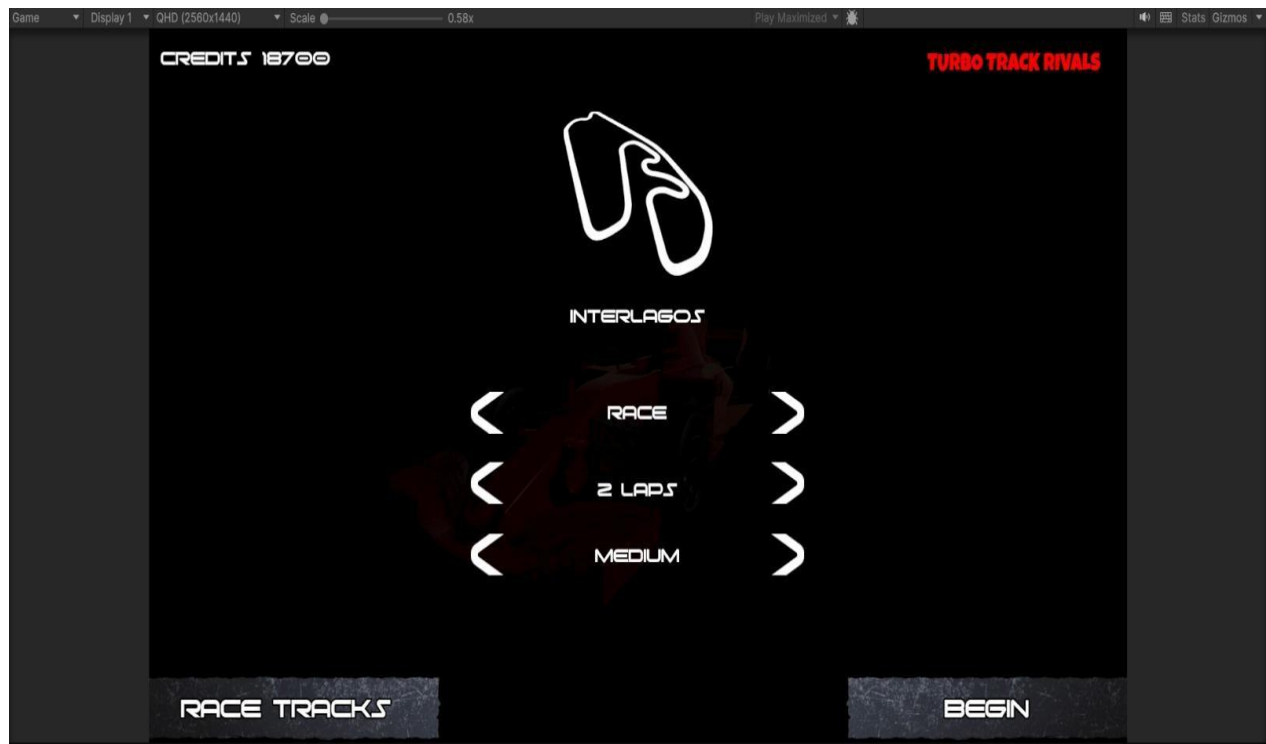


Figure 11 Custom Selection

Custom Selection screen also provides more control to the player allowing him to choose the **amount of laps** to complete in a race and the amount of **AI cars** they want to race. This also allows the players to adjust challenge and duration of each race to their liking, whether it is a short race with **less laps** and fewer competitors or a longer and tougher one with multiple competitors. Through this degree of customization, the game is guaranteed to be replayable and serve both the recreational player who wants to play shorter sessions as well as the competitive player who wants to engage in longer and high stakes races.

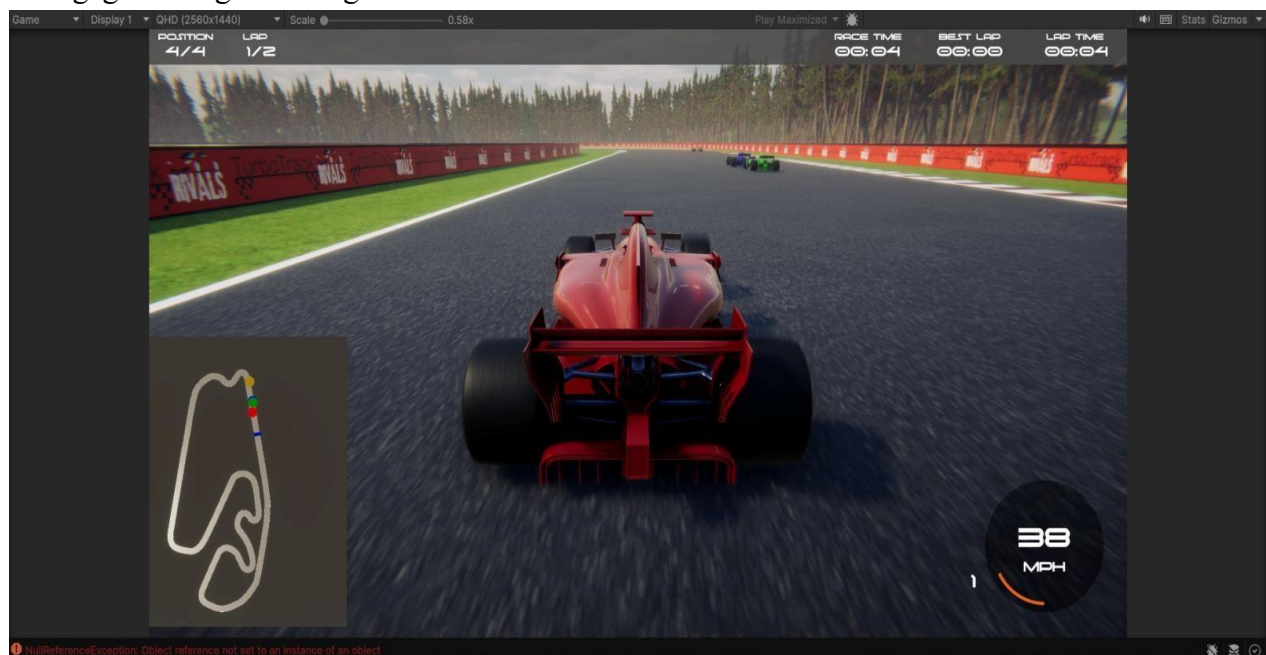


Figure 12 Game View

The **Game View** is the main screen of *Turbo Track Rivals*; it is where the real racing has been organized. In this perspective, players are able to drive their cars, follow the track, and compete with artificial intelligence that will aim to finish the race at the first position. It features the basic mechanics of the game as acceleration, braking, steering, and drifting, as well as the real-time feedback delivered through the **HUD** that displays all important data, such as the **lap count**, position, timer, and speed. This perspective is the core of the gamer experience of both the thrill of racing at high speed as well as being plunged in the challenge of beating their opponents.

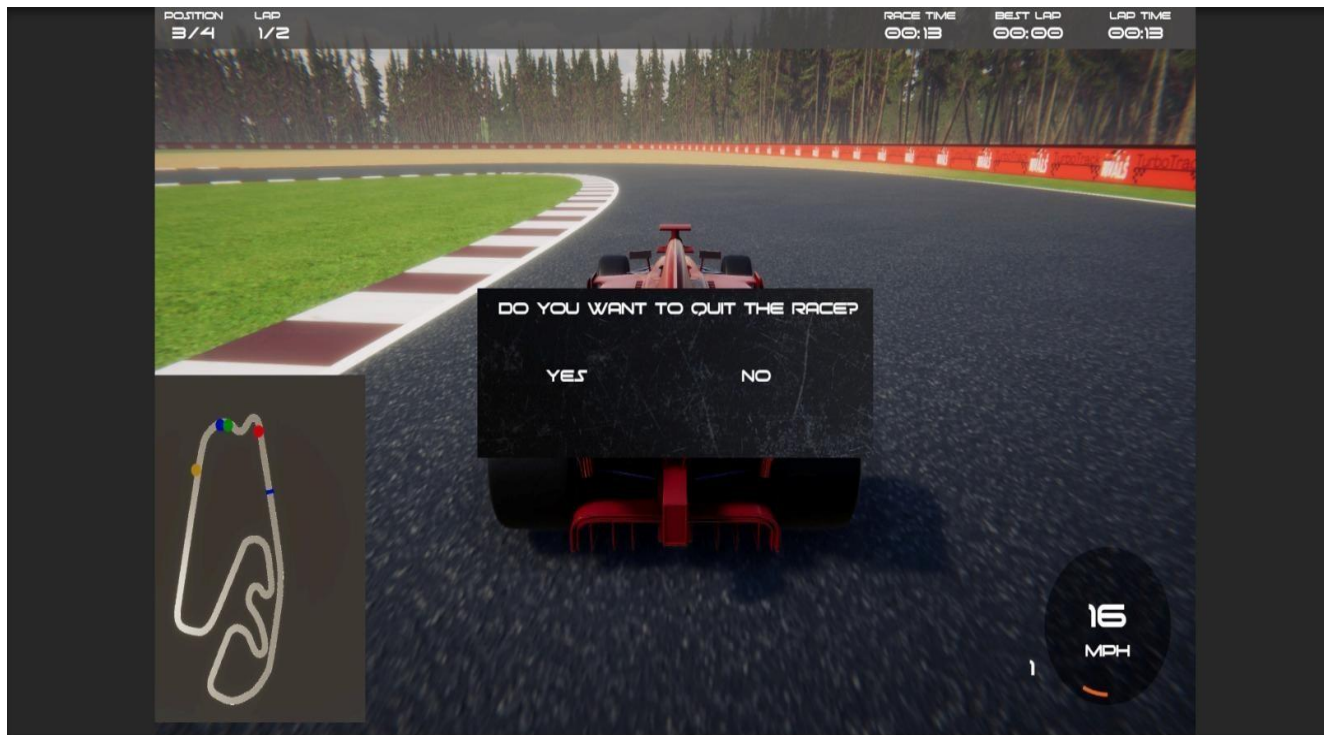


Figure 13 Exit Menu

Exit Menu allows a player to have a chance to exit the **Game View** in case they want to take a break or quit their session. It is in this menu that the players are able to choose to quit the game, or go back to carry the racing on with a seamless transition being made to the scene corresponding to the choice made. This option allows the players to be in charge of their experience as it is **flexible** and does not create any distraction to the gameplay. The **Exit Menu** offers **usability** and **convenience** by offering easily understandable and reachable options, and provides the players with the chance to control the sessions with ease.

The scene switching from the pause menu and exit menu provides the ability to players to have the freedom of pausing and exiting from the game any time, interactivity is followed throughout the game to make the users have best experience and seamless switching between different scenes. The exit tab is followed by pressing the escape key, which pops up the exit menu tab, through which the players when press the yes key will close the game and if pressed the no key then it will resume the game.

6. Testing Evaluation

The testing of Turbo Track Rivals was critical towards the general quality, stability and satisfaction of players. It was done by a series of **quality assurance** where several rounds of reviewing and refining were undertaken following the development milestones. This was mostly in the form of manual testing, where systematic test cases were used to thoroughly test the basic mechanics of the game, such as the responsiveness of vehicle control, proper collision detection, lap counting, the behavior of the AI opponents, and the power up feature.

Besides manual checks, there were **playtesting** meetings with users aimed at providing useful feedback on gameplay balance and usability as well as general enjoyment. This assisted in determining problems like confusing navigation, too aggressive AI, or vague UI components, which were then improved with the assistance of tester feedback. Another important area was performance testing, optimization being done on various tracks, race modes and environmental conditions that allowed the framerates to be smooth, loading times to be as short as possible, and responsiveness was always consistent even on **mid-range machines**.

Through coordinating structured test cases, actual user response, and more focused performance testing, the testing stage did not only assure the technical stability of the game, but also established that Turbo Track Rivals provided a playable, refined and dependable racing experience. This cyclical model provided a fast way of problem identification and solving, which laid a strong base on future improvement and scalability.

Table 6.1: Test Cases for Turbo Track Rivals

The test cases confirmed that all core gameplay mechanics and system functions of *Turbo Track Rivals* worked as intended. Functional requirements were validated through tests of vehicle acceleration, steering, braking, collision handling, lap counting, race completion, and AI navigation, all of which passed successfully. Non-functional requirements were also met, with smooth scene switching, stable pause/resume functionality, responsive UI elements, and consistent track-specific features across Stadium, Desert Dunes, and Snow Track. Overall, the results demonstrated that the game delivers reliable performance, intuitive controls, and a stable player experience across different modes and environments.

Table 7 Test cases

Test Case	Description	Expected Result	Result
Vehicle Acceleration	Player presses the acceleration key (e.g., W or up arrow)	Vehicle smoothly accelerates forward based on input and terrain	Pass
Steering and Braking	Player uses left/right keys and brake key	Vehicle responds with realistic turning and decelerates effectively	Pass
Obstacle Collision	Player hits a track obstacle (e.g., barrier, ramp)	Vehicle slows down or bounces with proper physics reaction	Pass

Lap Completion	Player crosses all checkpoints and completes a lap	Lap counter increments and lap completion sound is triggered	Pass
Race Completion	Player finishes the required number of laps	Race ends, scoreboard or result screen is shown	Pass
AI Opponent Navigation	AI car follows its track route	AI follows path using NavMesh or waypoints and competes realistically	Pass
Scene Switching	Player selects a track or game mode from main menu	Selected track or mode loads correctly with all assets initialized	Pass
Game Pause and Resume	Player presses pause (e.g., Escape key) during gameplay	Game pauses, and resumes correctly without affecting gameplay	Pass
UI Functionality	Player interacts with UI elements (Start, Quit, Restart, etc.)	Buttons perform correct functions and respond to clicks	Pass
Track-Specific Features	Each track has unique visual/obstacle settings	Stadium, Desert Dunes, and Snow Track load with appropriate environments	Pass

6.2 Integration of Game Components

The **integration phase** of *Turbo Track Rivals* was a pivotal stage in development, aimed at combining all individual modules into a cohesive and fully functional game system. The primary focus was on ensuring seamless interaction between critical components such as **vehicle control**, **AI opponents**, **user interface (UI)**, and **physics simulation**. Real time synchronization between player input, environmental triggers, AI decision-making, and on-screen feedback was essential to maintain both immersion and gameplay fluidity. It is such an important aspect of the game as it provides feedback to the players in real time also it enables the players to be capable of take decision based on their status in the race.

The **vehicle control** system was highly intertwined with the physics engine of Unity to provide **realistic acceleration**, **braking**, **drifting**, and **crash behavior** and at the same time maintain an approachable arcade feel. This had incorporated the effect of soil types like asphalt of stadium, desert sand, and icy snow on the handling directly. The supporters of the **AI** were linked to navigation grids, goalposts, and roadblocks of the track track, which allowed the opponents to change their behavior dynamically in response to the race environment. This not only resulted in the competition being unpredictable but it also generated a feeling of fairness and excitement with the AI drivers responding to player actions.

On the presentation side, the **UI elements** such speedometers, lap counters, race timers and result screens were directly connected to the underlying game logic. This enabled them to update instantly as events in the game took place e.g. passing checkpoints, maneuvering the vehicle

through hostile terrain or making laps. The **UI** offers a vivid presentation of all the required items that should be shown to the gamer to allow them to track their progress and understand what action to perform next and what tactics to use to beat other opponents.

In general, the integration process provided the creation of the harmonious functionality of all the modules, which supported the coherence of the game logic and provided the real-time **responsive feedback**. It was a holistic solution both in terms of enhancing performance across all the tracks and modes but also a more polished and immersive experience in which all the systems co-operated to facilitate seamless and enjoyable racing gameplay as well as giving a just playing field to all the new or amateurs and also to other players who are experienced gamers or interested in racing game.

Table 6.2: System Interactions in Turbo Track Rivals

Table 8 System interactions

System Interaction	Description	Expected Behaviour	Result
AI System + NavMesh	AI opponents navigate the track	AI follows waypoints using NavMesh and reacts to track layout and obstacles	Pass
Input System + Vehicle Controller	Player accelerates, steers, brakes	Vehicle responds accurately to input with realistic or arcade-like behavior	Pass
Vehicle System + Physics Engine	Car drives over terrain, hits ramps or barriers	Vehicle reacts with proper collisions, bouncing, or flipping as per physics rules	Pass
Lap System + SceneManager	Player completes all laps	Final race screen or next scene loads automatically	Pass
Timer System + UI Display	Race starts or ends	Countdown timer and race duration visible and updated in real time	Pass
Collision System + AudioManager	Car collides with barrier or opponent	Appropriate crash sound and effects are triggered	Pass
Checkpoints + Lap Tracker	Player passes through all checkpoints in order	Lap counter increments and lap completion feedback is triggered	Pass
Menu System + SceneManager	Player selects a track or game mode from the main menu	Game switches to the selected track or mode scene with all objects initialized	Pass

The *Turbo Track Rivals* system interaction testing involved the **verification** of the **interaction** between various modules and their role in providing the smoothness of the gameplay. The AI System and **NavMesh** was used to make sure that **AI opponents** used predetermined waypoints, and reacted to track layouts and obstacles dynamically. The Vehicle Controller and the Input

System were also tested to ensure that the actions of the players like accelerating, steering and braking were translated into the correct and responsive behavior of the vehicle. The Vehicle System and **Physics Engine** cooperated to give the simulation of the realistic interaction to the terrain, ramps, and barriers including the correct handling of collisions, bouncing, and flipping. The Lap System and **SceneManager** add-on ensured that when the lap was completed the race would proceed to the final results screen or would smoothly go to the next scene. The Timer System that included the **UI Display** was also tested to ensure there were proper countdowns when starting the race and real-time updates when playing the game. Likewise, the Collision System and **AudioManager** generated adequate sound and effect effects on vehicle collisions, which helped in the immersion. Checkpoints and Lap Tracker provided a good lap tracking as only after all the checkpoints were cleared, the lap would advance. Finally, the interaction between the Menu System and **SceneManager** proved that in case the player chose maps or game modes, the intended scene was loaded and all objects and assets were initialized correctly.

7. Play Testing and Beta Evaluation

Turbo Track Rivals playtesting was necessary in the process of quality adjustment and creation of a balanced and enjoyable game. More than **20 players**, both casual and professional, took part, and the project supervisor were experienced racers, which gave a wide range of perspectives.

Gameplay, AI behavior, visual clarity and performance were measured during the beta sessions. ableness, and difficulty balance. Testers provided feedback regarding how responsive vehicle controls were, how competitive **AI opponents** were and how readable tracks and HUD elements were. The performance testing also revealed the optimization points to ensure that the framerates and responsiveness stay smooth in every map.

Playtesting was used to detect some design inconsistencies, including inconvenient navigation, ambiguous **UI**, and AI beating players, which would be improved in subsequent iterations. It also guided enhanced track layouts and difficulty options, to keep the game interesting to both new players and high-end players. In general, playtesting was used as a quality control measure and player oriented. testing, making *Turbo Track Rivals* turn into a refined and realistic racing.

7.1 Testing Sessions Conducted

This is a description of the gameplay testing conditions organized around *Turbo Track Rivals* that was conducted throughout all tracks and the available **game modes**. The testing environments were well-crafted and tested to test various elements of the racing experience that included the **responsiveness of player control**, the **interaction of the vehicles with various environments and obstacles**, the **competitiveness and flexibility** of the AI, as well as the stability of the **UI feedback systems**, including lap counters, race timers, checkpoints, and result displays.

Along with examining specific mechanics, the overall race flow was also targeted, so that the segmentation between stages was without any issues, checkpoint identification was predictable, and the difficulty levels were balanced between the casual and advanced players. Parallel performance testing was done to ensure the stability of frame rates and the rapid loading of scenes

and the visual clarity test was done to ensure that the track readability and the on-screen indicators assist in effective decision-making of players.

These well organized testing conditions were critical towards defining the end quality of Turbo Track Rivals. They assisted in revealing the weaknesses of the design, in spotting the places where the level layouts needed to be refined, and in influencing the changes in the AI behavior to create fair but competitive races. Synchronization between essential parts of the game such as vehicle control, UI updates, lap tracking, scene transitions was also tested out. Due to this, the game became more stable, the performance of the game became smoother, and the players could feel that the game was more real and closer to the actual racing. These outcomes have much more than short-term gains as they end up giving a firm base on which to scale up to and it is easier to bring in new features and enhance the game mitigating the need of extra features.

Table 7.1: Play Testing Sessions

Table 9 Play testing sessions

No.	Test Environment	Player Focus Area	Purpose
1	Stadium Track	Speed control, obstacle handling	Evaluate player's ability to maneuver through tight curves and speed zones
2	Desert Dunes	Terrain variation, powerup collection	Test player response to sandy terrain, drifting etc
3	Snow Track	Slippery surface, AI overtaking	Assess difficulty due to low traction and AI navigation in icy conditions
4	Sprint Mode	Time-based challenge, player pressure	Test time-management under pressure and reaction speed
5	Circuit Mode	AI competition, lap tracking	Evaluate AI difficulty scaling, race pacing, and lap progression accuracy

Turbo Track Rivals has been tested in various environments and modes in order to test various areas of gameplay. The emphasis in the **Stadium Track** was based on speed regulation and obstacle control, and how players would navigate around narrow turns and through high velocity zone. The **Desert Dunes** setting focused on the variation of the terrain, with sandy surfaces and drifting as a factor that adversely affect the player control, as well as interaction and responses. The **Snow Track** presented slippery surfaces, low traction, which served the purpose of testing the adaptability of the player and overtaking and navigation of the AI in icy conditions. Time-based challenges were applied in Sprint Mode and measured the pressure handling, quick decision-making, and speed in reaction on a tight time constraint. Lastly, the **Circuit Mode** focused on **AI race and lap tracking**, which had to be difficult and provide fair pacing and proper lap progression between multiple racing rounds.

7.2. Feedback Themes Collected

The table below has identified and tabulated the motivating themes of most feedbacks obtained by the two testers of the play as well as the project supervisor during the evaluation sessions. The feedback emphasized mostly the most critical aspects of the game, including usability, which judged how simple it was to navigate the **menus and interfaces**; **gameplay balance**, which judged the fairness of competition between **AI opponents** and players; response to control, measuring how nice and accurate the gameplay was across various maps and game modes; visual clarity, such as the readability of the tracks and elements in the HUD; and the overall player experience, which judged how enjoyable and immersive the game experience was. Through structuring the feedback in such a manner, it is simplified to find the strengths, areas of improvement, and focus on improvements to be made in future versions of Turbo Track Rivals.

Table 7.2: Feedback

Table 10 Feedback

Feedback Area	Player Comments
Track Navigation	Some checkpoints were hard to spot; players suggested adding clearer visual markers
AI Difficulty	AI opponents were either too predictable or too aggressive on certain tracks
Vehicle Handling	Controls felt responsive, but a few players suggested improved drifting mechanics
UI Feedback	Players wanted more prominent lap counters and clearer race completion indicators
Supervisor Review	Recommended refining the transitions between game modes and improving menu flow

The **feedback table** shows a systematic analysis of the feedback and observations obtained during the playtesting and the reviews performed by the supervisors. The table has simplified the process of identifying patterns and common issues about which several participants raised their concerns by identifying the recurring themes, usability, gameplay balance, control response, visual clarity, and the overall engagement of the player. This organization does not only emphasize the strong points of Turbo Track Rivals but also identifies particular areas that require the most improvement.

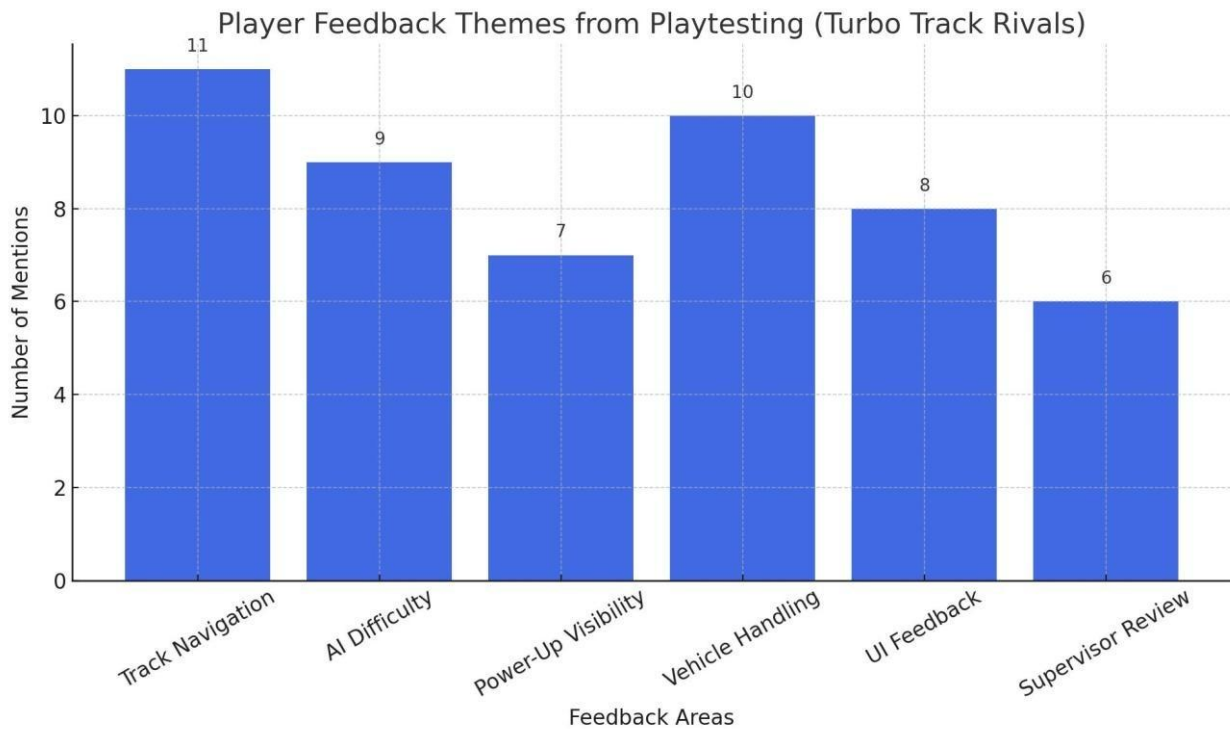


Figure 14: Feedback graph

To supplement the table, the **feedback graph** offers a graphical representation of this set of themes demonstrating the frequency and the level of emphasis of every category that testers discussed. This figure map will allow recognizing in a matter of minutes the elements of the game that received the most positive feedback and the points that received the most critical feedback. The table and the graph taken together offer a detailed and a high-level view of the evaluation process, which is why they can be useful in prioritizing refinements and future development choices.

7.3 Outcomes and Improvements

According to the useful suggestions obtained during the playtesting and internal quality assurance tests, a number of focus areas were identified to improve Turbo Track Rivals to create a more refined gameplay and a smoother player experience. The review of feedback revealed important issues related to direction of the player, artificial intelligence, responsiveness to the control, and understandability of user interface components. All these spheres were thoroughly examined and considered in the form of iterative corrections, which meant that, besides making the game more stable, it has become more entertaining and more appealing to a broad audience of players.

Taking the latter as an example, player guidance was improved by eliminating the ambiguity of track directions with more visual cues and checkpoints, and AI behavior was rebalanced to a more successful point. between competition and equity. The control systems in vehicles were refined. ability to be responsive on various terrains, less frustration and at the same time realistic. Also, UI elements like lap counters, race timers, and menus were redesigned to have enhanced visibility

and usability to ease interaction and improved feedback purposes in the races. The table below is a brief summary of the key problems that were found during testing and the corrective measures that were taken. These enhancements, combined together, greatly improved the quality and realness of the entire Turbo Track Rivals and gave the game a more immersive, balanced, and enjoyable racing experience.

Table 7.3: Precaution*Table 11 Precautions*

Issue Identified	Action Taken
Confusing track navigation	Introduced glowing checkpoints and directional arrows on tracks
Unbalanced AI overtaking behavior	Tweaked AI pathfinding and overtaking logic for fairer competition
Power-ups difficult to notice	Increased brightness, added animation effects to highlight active power-ups
Slippery vehicle control on snow	Adjusted grip and traction values for improved handling on icy surfaces
UI lap counter unclear	Enlarged lap indicators and used contrast-friendly colors for better visibility
Mode switching delay	Optimized transition speed between Sprint and Circuit modes

The precaution table is an organized summarization of the most serious problems detected in the course of testing and assessment of Turbo Track Rivals, as well as the solutions that will be taken to overcome them. The table provides a case of each issue that impacted the quality of the gameplay, including getting lost on the track, unbalanced AI catching up, or insufficient lap counters, and the description of the very measures that have been made to enhance the playing experience of the players. As an example, the interactive elements were added to the game, such as glowing checkpoints and directional arrows, the AI pathfinding was also optimized to provide a more even competition, and grip values on snowy roads were increased to allow a easier control. In the same way, the clarity of UI was increased with the enlarging of lap indicators, power-ups were made more visible with brightness and animations, or mode transition delays were minimized through optimization of the scenes loading. This table is a record of the solutions used, but also demonstrates how the development team is dedicated to addressing the concerns of the players systematically, providing a constant improvement of the solution and better standards of gameplay quality.

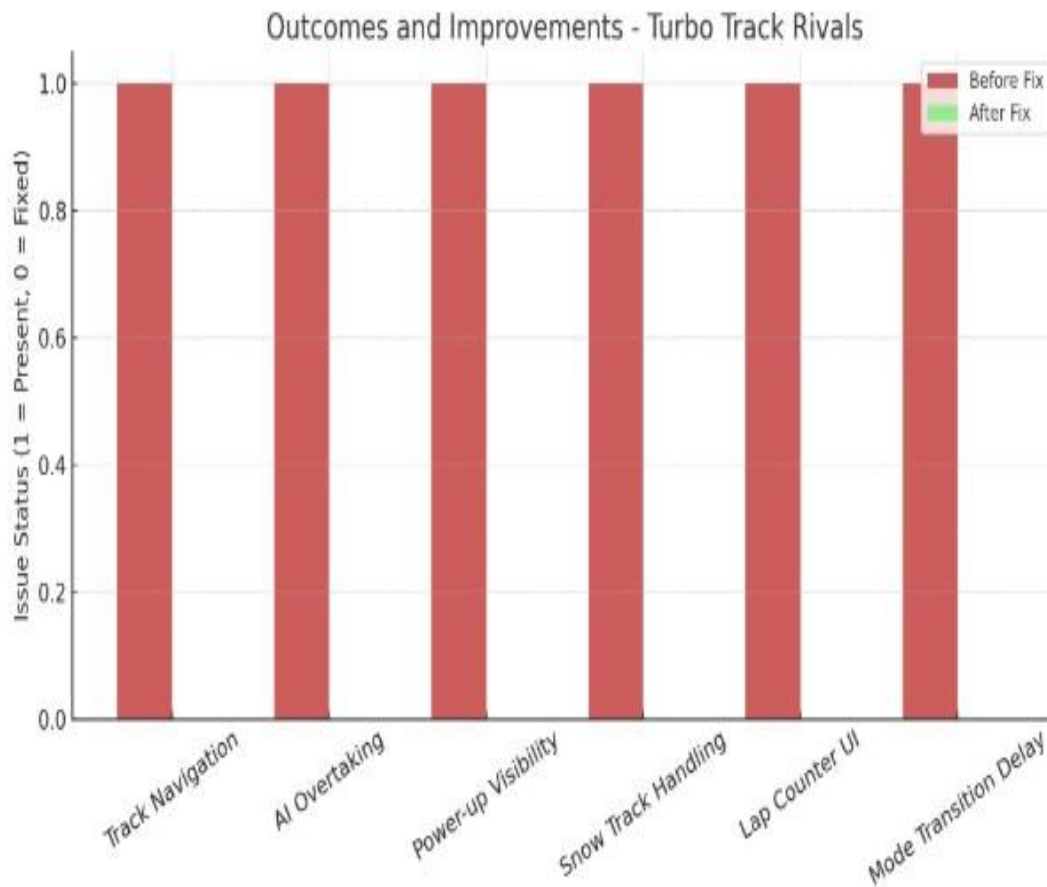


Figure 15: Precaution graph

To supplement the feedback table, a precaution graph was created in a manner that the issues that will arise during playtesting, and the corrective measures required to fix these issues will be graphically displayed. The chart shows that every enhancement is directly related to the problem area, which may be the navigation in tracks, the imbalanced behavior of AI, or non-clear elements of the UI. Mapping the problems with their solutions, the precaution graph also proves the responsiveness of the development process as well as presents the proactive measures which should be taken to avoid the problems of the same nature in the next round. Such visual effect facilitates easier appraisal of the general stability and polish of Turbo Track Rivals, as well as a source of benchmark on quality in future-released updates.

8. Conclusion and Future Work

The sections below will describe in detail the results of the creation of Turbo Track Rivals and its strengths, its main features, and the overall successful implementation of several design and technical aspects. Moreover, these parts also provide a possible development plan of the future improvements that might make the gameplay more polished, add more content, and introduce new features. This prospective system provides the game with not only the entertaining experience it currently offers but also a great potential of keeping the players interested in the game in the long-term, replay it and scale it to the new user requirements and the industry trends and conditions.

8.1 Conclusion

Turbo Track Rivals is the end result of a passionate game development process that combines technical accuracy with the design of an imaginative game. The project was created as an immersive racing game in Unity 3D and C# and in a modular iterative development process, providing a rapid-paced immersive combination of realistic vehicle physics and arcade-style racing. The game has three distinct themed tracks which are **Stadium**, **desert dunes** and **snow track** that are meant to provide different environments and challenges.

The game has been developed with feedback-led changes, constant playtesting, and several development sprints to deliver a responsive, exciting, and highly replayable racing experience to players. Featuring two fundamental game modes Sprint Mode (time trial) and Circuit Mode (competitive AI race), *Turbo Track Rivals* provides dynamic gameplay to both the casual and competitive players. **Car physics**, **AI behaviors** and **immersive audio-visual effects** all make it an enjoyable experience that is neither too realistic, nor too arcade-based. The project has achieved the following major achievements: Creating a balanced racing experience through technical accuracy and interactivity. To accomplish this, three different race environments have been made that have different terrains, obstacles, and track layouts to force the player to change their driving style and strategies.

More realism is contributed by the inclusion of sophisticated vehicle physics, so that the acceleration, braking, traction, and collision detection systems are carefully tuned to produce the behavior of a real car, but still making the game retain the fun and fast-paced arcade racer feel. To accompany this is the incorporation of custom-written **AI opponents**, which are directly programmed to dynamically react to the conditions of the track, position, or location of a player, and in-game factors like power-ups. This will make all the races feel different to create a fresh and challenging case. Outside the basic mechanics, much focus was placed on visual and gameplay polish in which the details of the environment and sound effects were considered and the aesthetics of the tracks were enhanced as suggested by the play testers to enhance immersion and its general playability. **UI/UX improvements**, such as lap counters that are intuitive, an accurate race timer and informative post-race result screens, which give players clear and responsive feedback were equally important.

A combination of these attributes gives *Turbo Track Rivals* a balance of both technical complexity and player fun, as it is accessible to the non-gamer and competitive to those with a higher skill level of racing. It can be noted that *Turbo Track Rivals* was successfully delivered because the programming, design, testing, and iteration were closely coordinated. The development team continued to be user-focused, **performance-oriented** and **aesthetically appealing** so that the resultant product is both strong and pleasant to use.

8.2 Future Work

Although *Turbo Track Rivals* manages to provide a tremendously satisfying and overall entertaining experience of a single-player arcade racer, it also has so much potential of expansion and development. The game can be developed into a more refined and commercially viable racing game with its solid background. The underlying architecture has been done thoughtfully keeping in consideration the modularity so that new features and additions are easily accommodated without interfering with the existing mechanics. This will provide future opportunities to improve it with, more multiplayer modes, more tracks, more **sophisticated AI systems**, and **polished visual effects**, which would further enrich the experience of the overall player.

8.2.1 Proposed Enhancements

Potential future improvements include:

1. **Multiplayer Mode**

Introduce real-time competitive multiplayer via LAN and online matchmaking so players can race friends or global opponents. This will increase replayability and community engagement by adding lobbies, matchmaking, friend invites, and ranked/unranked match types. Implementation considerations include choosing an authoritative netcode model (to prevent cheating), reconciling latency with interpolation/rollback techniques, and providing simple UI flows for joining and creating matches.

2. **Car Customization System**

Add both aesthetic and performance customization: paint jobs, decals, rims, visual skins, plus upgradeable parts like engines, nitro systems, and suspension tuning. Cosmetic options let players express themselves; performance upgrades introduce long-term goals and strategic choices. The system should balance visual-only items vs. gameplay-affecting upgrades, integrate with the in-game economy, and expose straightforward UI in a garage environment.

3. **Dynamic Weather and Track Effects**

Implement real-time weather (rain, snow, fog) and track surface changes that affect traction, visibility, and handling. These dynamic conditions increase challenge and variety—forcing players to adapt strategy and vehicle setup. Technical notes: tie weather to physics/material parameters, add particle/audio effects for immersion, and include performance fallbacks for lower-end hardware.

4. **Career Progression System**

Create a structured long-term progression loop with unlockable tracks, achievements, driver rankings, sponsorship deals, and in-game currency rewards. A career mode motivates continued play and provides a narrative or meta-goals beyond single races. Design the progression to reward skill and exploration, ensure save persistence, and provide clear feedback on advancement and rewards.

5. **AI Difficulty Levels and Personalities**

Enhance AI by giving drivers distinct personalities (aggressive, defensive, opportunistic) and scalable difficulty settings. Parameterize behavior trees or state machines so opponents behave believably and scale to the player's skill. This improves replay value and ensures both new and experienced players find the races appropriately challenging.

6. **Power-Up Expansion**

Introduce new power-ups—EMP blasts to disable opponents briefly, temporary shields to block attacks, speed traps to slow racers, etc.—to diversify race strategies. Each power-up should have

clear telegraphing, balanced spawn mechanics, and counterplay options so they add depth without feeling unfair. Consider UI icons and audio cues so players can react quickly.

7. Split-Screen Mode

Add local multiplayer with split-screen support for couch play. This requires rendering multiple viewports, per-player input handling, and UI adjustments for smaller screen regions. Because split-screen multiplies rendering cost, include optimization paths (reduced draw distance, LODs) to maintain performance on target hardware.

8. In-Game Economy and Garage System

Build a garage where players can manage vehicles, spend earned currency on upgrades or cosmetics, and unlock new models. The economy should balance progression speed so players feel rewarded but not gated. Optionally support microtransactions only if balanced ethically; otherwise focus on earnable content and clear storefront UX.

9. Mobile and Cross-Platform Support

Optimize the game for Android, iOS, and consoles to broaden the player base. This entails input adaptation (touch controls, controller mappings), memory and battery/performance optimizations, platform-specific builds, and possible cloud-save / cross-progression support. Cross-play considerations and certification requirements should be planned early.

10. Global Leaderboards and Time Trials

Add online leaderboards and ghost-race functionality so players can compare lap times globally and race against top runs. Provide leaderboard filters (region, friends, platform), anti-cheat protections, and periodic leaderboard events or seasons to encourage competition.

11. Track Editor and Community Sharing

Offer a user-friendly track editor and a sharing system where players can create, upload, and download community tracks. Include validation tools to prevent performance-killing designs, moderation/flagging systems, and rating mechanics so the best community creations rise to the top great for longevity and user-generated content.

12. Narrative Racing Campaign

Develop a story-driven campaign with missions, rival characters, cutscenes, and voice lines that take players through an arc (e.g., rising through underground circuits to championship). Narrative elements can deepen player investment and serve as a scaffold for unlocking cars, sponsors, and unique tracks.

13. Localization and Accessibility Features

Expand reach with multilingual support, UI scaling, remappable controls, color-blind palettes, subtitle options, and adjustable difficulty presets. Accessibility features should also include clear audio cues, high-contrast HUD modes, and options for alternative input devices so more players can enjoy the game.

With these additions, *Turbo Track Rivals* is well-positioned to evolve from a student project into a high-quality racing title that can compete in indie and commercial markets. Continued development, community engagement, and platform expansion can transform it into a standout experience for racing fans globally.

9. References

Help and inspiration were taken from the following books, websites, tutorials, and platforms during the development of *Turbo Track Rivals*:

1. <https://learn.unity.com/>
2. <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/>
3. <https://assetstore.unity.com/>
4. <https://www.youtube.com/c/Brackeys>
5. https://en.wikipedia.org/wiki/Need_for_Speed
6. <https://free3d.com/>